h_da
HOCHSCHULE DARMSTADT
UNIVERSITY OF APPLIED SCIENCES

FACULTY OF ELECTRICAL ENGINEERING
AND INFORMATIONS TECHNOLOGY

An Introduction to

Hardware Description Language

VHDL

Prof. Dr. T. Schumann

Overview:

- **The Modelling Process**
- **Objects, Types, Operators, Delay**
- **Multiple Architectures, Concurrency, Iterations**
- **Modelling Styles, Sequential Operations, Variables**
- **Sequential Control and Attributes**
- **Conditional Assignment, Concatenation**
- **Arrays, Loops, Assert**
- **Components**
- **Test Benches**

Advanced Topics:

- **Functions and Procedures, Blocks**
- **Textual inputs and outputs**
- **Types and Type Conversion**
- **Packages, Libraries**

Simulation tool used in lab:

ModelSim – Intel FPGA Starter Edition
(free download from www.intel.com)

FPGA Synthesis tool used in lab:

Xilinx Vivado WebPACK
(free download from www.xilinx.com)

Tutorials:

ModelSim tutorial (pdf-file)
Vivado Design Suite User Guide (pdf-file)

The Modeling Process

Model Development

VHDL Model Structure

Entity, Architecture

Commenting Code

Model Testing

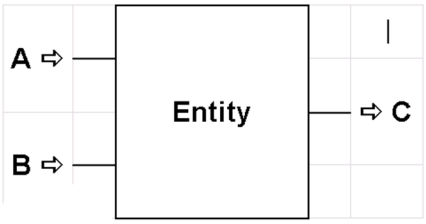
Signal Assignment

Model Development

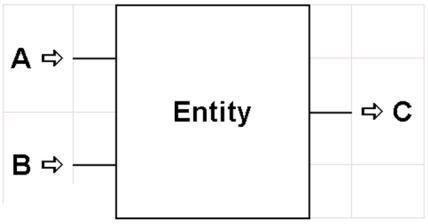
	**Compile**	
Debug		**Debug**
	**Simulate**	

Specification

- ***Create an entity that receives two digital signals and that outputs a single signal.***
- ***If both input signals are low, the output signal is to be high.***
- ***For any other combination of high or low inputs, the output is to be low.***



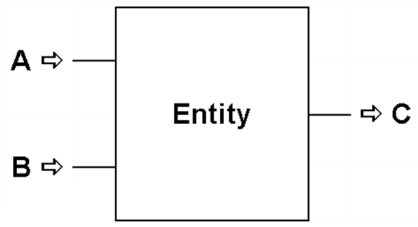
Analysis



Truth Table:

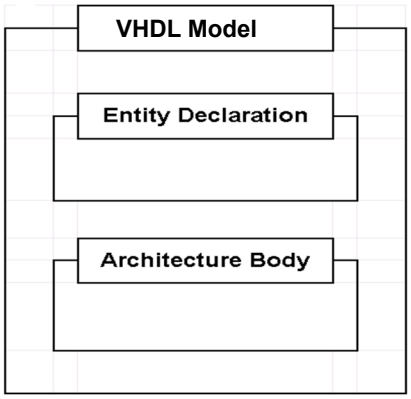
Inputs		Output
A	B	C
0	0	1
0	1	0
1	0	0
1	1	0

Design



- **Basic modelling unit is called a Design Entity**
- **It may represent an entire system, a board, a chip, or a gate**

Model Structure



- **VHDL Models consist of an Entity Declaration and an Architecture Body**

Entity Declaration

Names an entity and defines an interface between the entity and its environment

Format:

```
ENTITY entity_name IS
   [ GENERIC ( generic_list ) ; ]
   [ PORT ( port_list ) ; ]
END [ entity_name ] ;
```


```
ENTITY nor_gate IS
  PORT ( a, b : IN  BIT ;
         c : OUT BIT ) ;
END nor_gate;
```

Port Statement

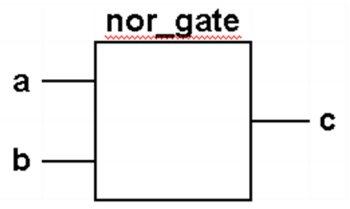
- **Identifies ports used by the design entity to communicate with its environment**

- **Format:**

```
PORT ( name_list : mode type ;
                .
                .
                .
       name_list : mode type ) ;
```


Port Statement

- **Example**



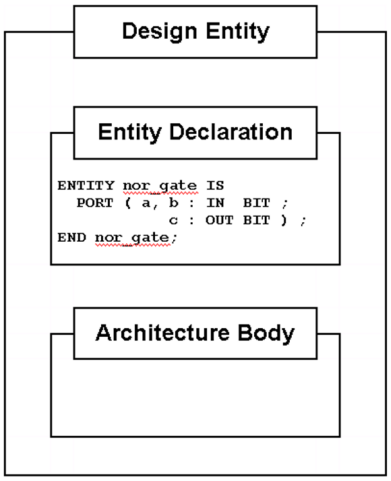

```
PORT ( a, b : IN  BIT ;
       c : OUT  BIT ) ;
```

Port Mode

- **Identifies direction of data flow through the port**
- **All ports must have an identifier mode**
- **Allowable modes:**

⇨□	IN	-Flow is into design entity
⇦□	OUT	- Flow is out of design entity
⇔□	INOUT	Flow may be either in or out
⇔□	BUFFER	- In or out, limited to one 'in' source
□	LINKAGE	-Documentation only, does not pass signal information, cannot be read or written

nor_gate Entity Declaration



Architecture Body

- **Establishes relationship between inputs and outputs of a design**

- **Format:**
  ARCHITECTURE body_name OF entity_name IS
  -- declarative_statements
  BEGIN
  -- activity_statements
  END [body_name];

- **Example:**
  ARCHITECTURE data_flow OF nor_gate IS
  BEGIN
  -- activity_statements
  END data_flow ;

Commenting Code

- **A double hyphen (--) indicates that all material from that point to the end of the line is to be treated as a comment**

- **Example:**

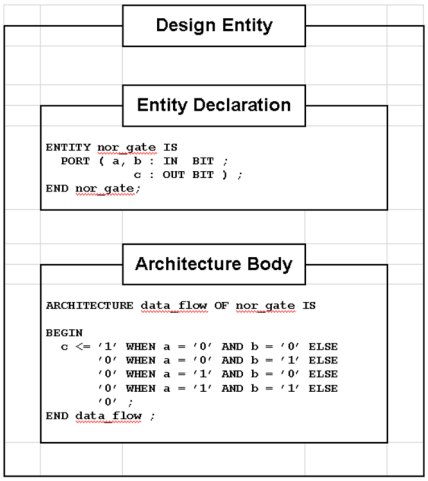
```
ARCHITECTURE example OF comments IS
   -- Area before begin
   -- is declarative area
BEGIN
   -- Area after begin contains
   -- statements that have activity
END data_flow ;
```

nor_gate Architecture Body

Inputs		Output
a	B	c
0	0	1
0	1	0
1	0	0
1	1	0

- **ARCHITECTURE data_flow OF nor_gate IS**
  BEGIN
  c <= ’1’ WHEN a = ’0’ AND b = ’0’ ELSE
  ’0’ WHEN a = ’0’ AND b = ’1’ ELSE
  ’0’ WHEN a = ’1’ AND b = ’0’ ELSE
  ’0’ WHEN a = ’1’ AND b = ’1’ ELSE
  ’0’ ;
  END data_flow ;

nor_gate Model



Creating Source Code

- **VHDL source code may be placed in any directory**
- **Source code may be entered with your choice of text editor or with a tool- specific VHDL Editor**
- **VHDL source object name can be any name**

  Recommendation:

  Use the name of entity, architecture, package, or configuration in the source object name.

Compiling VHDL Code

- **VHDL compile may be invoked from within Simulation Tool (ModelSim) from the menu or by command line entry**

  In ModelSim a VHDL Editor window is available

Model Testing

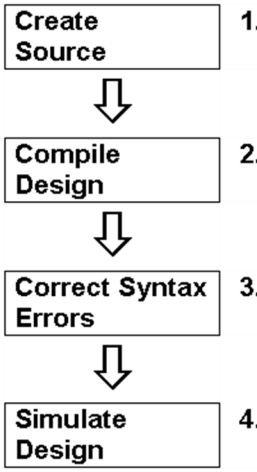
- Done within a simulator
- Invoke simulator from command line
- Simulator includes line-by-line VHDL debugging

```
1   ENTITY nor_gate IS
      PORT ( a, b : IN  BIT ;
             c : OUT BIT ) ;
    END nor_gate;


    ARCHITECTURE data_flow OF nor_gate IS

    BEGIN
      c <= '1' WHEN a = '0' AND b = '0' ELSE
           '0' WHEN a = '0' AND b = '1' ELSE
           '0' WHEN a = '1' AND b = '0' ELSE
           '0' WHEN a = '1' AND b = '1' ELSE
           '0' ;

    END data_flow;
```


nor_gate Model Test

- Invoke the simulator on the nor_gate model
- Set simulator to trace signals a, b, and c
- Within the simulator set the stimulus as follows:
  force a 0 0
  force b 0 0
  force a 1 50
  force b 1 100
  force a 0 150
  force b 0 200
- Run the simulation as follows:
  run 250

VHDL Model Test



1. Create VHDL source using plain text editor
2. Compile the source code either from a command line or within Modelsim
3. Correct and syntax errors found and recompile source
4. Invoke the simulator (Load Design) and apply stimulus to the design

Signal Assignment

- **Assignment of waveform to a signal object**
- **Assignment operator (<=) is the only operator that can be used to assign waveforms to signals**
- **Format:**

  target_object <= waveform ;

- **Example:**

  signal_a <= '0';

Conditional Signal Assignment

- **Assigns waveform to signal object based on the validity of a condition**
- **Format:**

```
target_object <= waveform WHEN condition
                 ELSE
                 waveform ;
```

- **Condition must produce a boolean value**
- **Example:**

```
signal_a <= '0' WHEN b = '1' ELSE
            '1' WHEN b = '0' ELSE
            'X' ;
```


Selected Signal Assignment

- **Assigns waveform to signal object based on selector value**
- **Format:**

```
with <selector> select
  target_object <= waveform WHEN <selectorvalue_1>,
                   waveform  WHEN <selectorvalue_2>,
                   waveform WHEN others;
```

- **Example:**

```
with b select
     signal_a <= '0' WHEN  '1',
                 '1' WHEN '0',
                 'X' WHEN others;
```

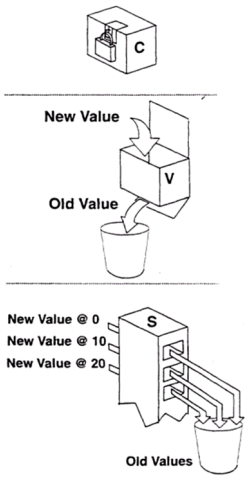
Objects

- Containers for values
- Classes:
  - Constants

    Objects assigned an initial value that cannot be changed
  - Variables

    Objects assigned a single value that may be changed
  - Signals

    Objects assigned values with associated time factors that may be changed



Declaring Objects

- **Declaration format:**

  OBJECT_CLASS identifier : TYPE [:= init_val] ;
- **Examples:**
  - **Constants**
    CONSTANT delay : TIME := 10 ns;
    CONSTANT multiplier : REAL := 5.25 ;
  - **Variables**
    VARIABLE sum : REAL ;
    VARIABLE voltage : INTEGER := 0 ;
  - **Signals**
    SIGNAL clock : BIT ;
    SIGNAL inner_works : std_ulogic := 'X' ;
- **Objects in port statement are classified as signals by default**

Naming Objects

- 1987/1993 VHDL Basic Identifier Naming Rules
  - Valid characters:
    alpha characters (a - z)
    numeric characters (0 - 9)
    underscore (_)
  - Must start with an alpha character
  - May consist of any number of alpha, numeric, or underline characters
  - Underscore must be preceded and followed by alpha or numeric characters
  - Names are not case sensitive
- 1993 VHDL Extended Identifier Naming Rules
  - Any characters (except carriage return) surrounded by backslashes (\) are accepted as a literal identifier
    Examples: _text\ \$text\ \8-bit\ \8-Bit\ \8-BIT\

Initial Value

- Value assumed by object at time of declaration
- Appears after TYPE in declaration
- Declaration format:

  OBJECT_CLASS identifier : TYPE [:= init_val]
- Example:

  VARIABLE voltage : INTEGER := 0 ;
- If not assigned, system assumes a default value equal to the left-most or minimum value for type

Data Types

- **Identify set of values an object may assume and operations that may be performed on it**

  Classifications:

  – **Scalar**

    Numeric, enumeration, and physical objects

  – **Composite**

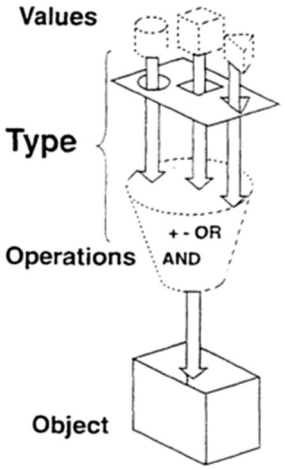
    Arrays and records

  – **Access**

    Value sets that point to dynamic variables

  – **File**

    Collection of data objects outside model



Scalar Examples

- **Numeric**

```
TYPE integer IS RANGE -214748348 TO 214748347 ;
TYPE real IS RANGE -1.79769E308 TO 1.79769E308 ;
```

- **Enumeration**

```
TYPE bit IS ('0','1') ;
TYPE Boolean IS (false, true) ;
```

- **Physical**

```
TYPE time is RANGE -9223372036854775808 TO
                   9223372036854775807
UNITS
  fs;                -- femtoseconds
  ps  = 1000 fs ;    -- picoseconds
  ns  = 1000 ps ;    -- nanoseconds
  us  = 1000 ns ;    -- microseconds
  ms  = 1000 us ;    -- milliseconds
  sec = 1000 ms ;    -- seconds
  min = 60 sec  ;    -- minutes
  hr  = 60 min  ;    -- hours
END UNITS ;
```

Predefined Types

- Certain scalar data types are predefined and do not require a type declaration statement
- Examples
  - boolean
  - bit
  - integer
  - real
  - character
  - time
  - severity_level

Accessing Commonly Used Types

- Sharing of type declarations is done through constructs known as packages
- Common types, procedures, and functions are located in packages such as std_logic_1164 (IEEE standard 1164-1993)
- LIBRARY makes selected library containing desired packages "visible" to a model
- USE clause makes packages visible to a model
- Example:

```
LIBRARY ieee ;
USE ieee.std_logic_1164.ALL ;
```

Using std_logic_1164

- **std_logic_1164 is the package standardized by the IEEE that represents a nine-state logic value system**

```
TYPE std_ulogic IS
  ( 'U', 'X', '0', '1', 'Z', 'W', 'L', 'H', '-' ) ;

TYPE std_logic IS
   ( 'U', 'X', '0', '1', 'Z', 'W', 'L', 'H', '-' ) ;
```

- **Example:**

```
LIBRARY ieee ;
USE ieee.std_logic_1164.ALL ;

ENTITY xor_gate IS
  Port ( a, b :  IN std_ulogic ;
            c : OUT std_ulogic ) ;
END xor_gate ;


ARCHITECTURE data_flow OF xor_gate IS

BEGIN
  c <= '0' WHEN a = '0' AND b = '0' ELSE
       '1' WHEN a = '0' AND b = '1' ELSE
       '1' WHEN a = '1' AND b = '0' ELSE
       '0' WHEN a = '1' AND b = '1' ELSE
       'X' ;
END data_flow ;
```


Using std_logic_1164

-
```
TYPE std_ulogic is (
 'U', --Uninitialized state    Used as a default value
 'X', --Forcing Unknown Bus contentions, error cond., etc.
 '0', --Forcing Zero   Transistor driven to GND.
 '1', --Forcing One    Transistor driven to VCC.
 'Z', --High Impedance 3-state buffer outputs
 'W', --Weak Unknown   Bus terminators
 'L', --Weak Zero      Pull down resistor
 'H', --Weak One       Pull up resistor
 '-'  --Don't Care  used for synth. and advanced modeling
                   ) ;
-----------------------------------------------------------
```

Type std_logic is defined with same nine-state logic values, but can be used to resolve bus conflicts if for example one sender applies a '1' and another sender applies a '0' to the same bus.

Operators

- Object type also identifies the operations that may be performed on an object
- Operators defined for certain predefined types:
  - Adding Operators:
    +, -, &
  - Multiplying Operators
    *, /, MOD, REM
  - Sign:
    +, -
  - Shift Operators*:
    ROL, ROR, SLA, SLL, SRA, SRL
  - Miscellaneous Operators:
    **, ABS, NOT
  - Relational Operators:
    =, /=, <, <=, >, >=
  - Logical Operators:
    AND, OR, NAND, NOR, XOR, XNOR*
- *- IEEE Std 1076-1993, Standard VHDL Language Reference

Logical Operators

- Are predefined for types bit and boolean
- Follow conventional definitions
  - AND
  - OR
  - NAND
  - NOR
  - XOR
  - NOT
- (XNOR - IEEE Std 1076-1993, *Standard VHDL Language Reference Manual*)

xor_gate Logic

- **An architectural solution for simple gate functions can be written using logical operators**
- **Example:**
  ARCHITECTURE logic OF xor_gate IS

  BEGIN
  c <= a XOR b ;
  END logic ;

xor_gate Model

```
LIBRARY ieee;

USE ieee.std_logic_1164.ALL;

ENTITY xor_gate IS

  PORT ( a, b:  IN std_ulogic ;

            C: OUT std_ulogic ) ;

END xor_gate ;

ARCHITECTURE logic OF xor_gate IS

BEGIN

  C <= a XOR b ;

END logic ;
```

Delay

- **Time period between cause and effect**
- **Delay selection:**
  - **Inertial**
  - **Transport**
- **Internal delay:**
  - **Delta**

Inertial Delay

- **Characteristic of any system where there is a time lag between cause and effect**

Example of un-delayed model:

```
c <= a AND b ;
```

Time Period	a	b	C
0	0	0	0
1	1	1	1
2	0	1	0
3	1	1	1
4	1	0	0

Example of delayed model:

```
c <= a AND b AFTER
one_time_period ;
```

Time Period	a	b	C
0	0	0	0
1	1	1	0
2	0	1	1
3	1	1	0
4	1	0	1

After Clause

- **Inertial delay is modelled with after clause**
- **Format:**

```
new_signal <= signal_expression AFTER time;
```

- **Examples:**

```
a_signal <= b_signal AFTER 10 ns ;

d_out <= '1' AFTER 10 ns,
         '0' AFTER 20 ns,
         '1' AFTER 30 ns,
         '0' AFTER 40 ns ;

q <= '1' AFTER 10 ns WHEN b = '1' ELSE
     '0' AFTER 5 ns ;
```


Delayed xor_gate Model

```
LIBRARY ieee;
  USE ieee.std_logic_1164.ALL;

ENTITY xor_gate IS
    PORT ( a, b:  IN std_ulogic ;
              c: OUT std_ulogic ) ;

END xor_gate ;

ARCHITECTURE logic OF xor_gate IS

BEGIN
    c <= a XOR b AFTER 10 ns;
  END logic ;
```

Signal Duration

- **Signals whose duration is less than delay specified in after clause are ignored**
- **Example:**
  Signal Assignment Statement:

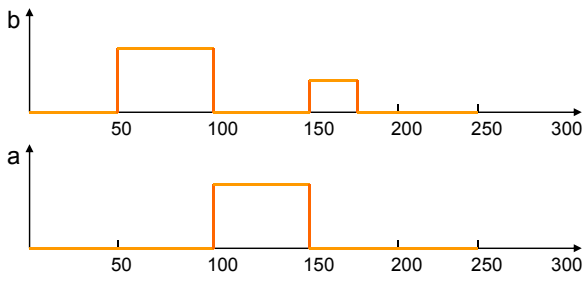
```
a <= b AFTER 50 ns ;
```

```
Stimulus Set:
force b 0 0
force b 1 50
force b 0 100
force b x 150
force b 0 175
run 250
```

Result:
25 ns unknown pulse on b at 150 ns is not passed to signal a

Signal Duration

- **Waveforms:**



Transport Option

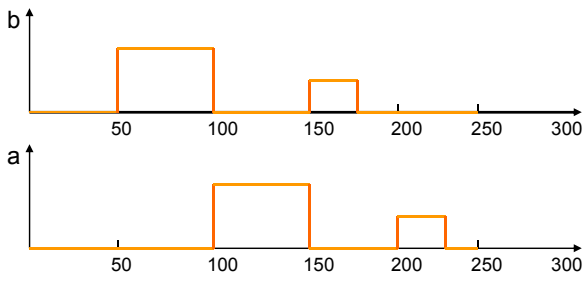
- **Passes signals regardless of duration**
- **Used when signals are delayed but 'filtering' effect produced by after clause unwanted**
- **Example:**
  Signal Assignment Statement:

```
a <= TRANSPORT b AFTER 50 ns ;
```

```
Stimulus Set:
force b 0 0
force b 1 50
force b 0 100
force b x 150
force b 0 175
run 250
```

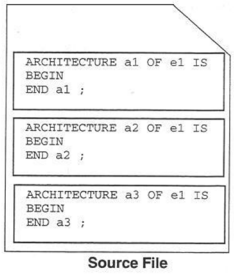
Result:
25 ns unknown pulse on b at 150 ns is passed to signal a

Transport Option



Multiple Architectures

- More than one architecture may be placed in model source file
- Each architecture must have a different identifier
- Only one architecture per instance may be identified for use during simulation

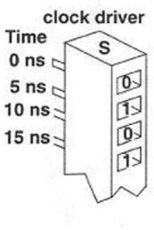


Signal Drivers

- Containers for projected waveforms of signals
- Created when a signal assignment is made
- Example

--filling a driver

```
Clock <= '0' ,
         '1' AFTER 5 ns ,
         '0' AFTER 10 ns ,
         '1' AFTER 15 ns ;
```



Filling Drivers

- **Negative time is not allowed**
- **Assignments within a single assignment statement must be made in ascending time order**
- **Example:**

```
Clock <= '0' ,
        '1' AFTER 5 ns ,
        '0' AFTER 10 ns ,
        '1' AFTER 15 ns ;
```


Filling Drivers

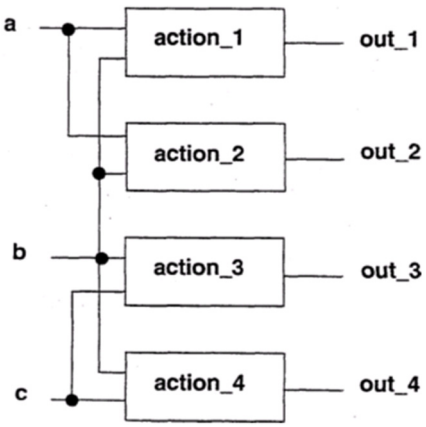
- **Assignments to a single signal by multiple concurrent statements create multiple drivers which must be "resolved"**
- **Example**

```
signal_out <= signal_in_1 AFTER 10 ns ;
signal_out <= signal_in_2 AFTER 10 ns ;
```

- **Multiple executions of assignment statement modify projected waveforms in driver**

Concurrency

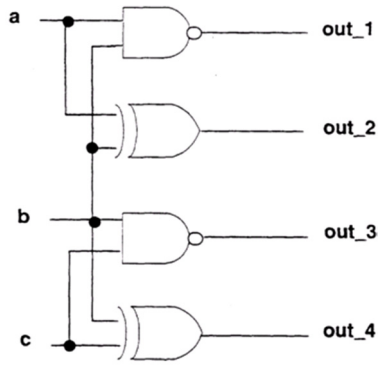
- Occurrence of two or more events in same time period
- In the following system, a change in signal "b" may cause a concurrent change in all the output signals



Concurrent Execution

- To model reality, VHDL processes certain statements concurrently

```
Example:
ARCHITECTURE example OF
concurrent IS
BEGIN
  out_1 <= a NAND b ;
  out_2 <= a XOR  b ;
  out_3 <= c NAND b ;
  out_4 <= c XOR  b ;
END example ;
```



Concurrent Statements

- A model's architecture is made up of one or more concurrent statements
- Concurrent Statements:
  - Block Statement
  - Process Statement
  - Assertion Statement
  - Signal Assignment Statements
  - Procedure Calls
  - Component Instantiations
  - Generate Statements
- Each concurrent statement represents a unit of functionality

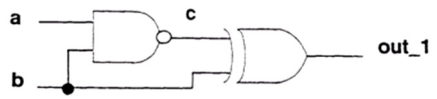
Statement Activation

Signals connect concurrent statements

Concurrent statements activate by event on a signal "entering" the statement

Example:

```
ARCHITECTURE example OF concurrent IS
  SIGNAL c: std_ulogic ;
BEGIN
  c     <= a NAND b ;  --nand_gate
  out_1 <= c XOR  b ;  --xor_gate
END example ;
```



An event on a signal a activates the nand_gate statement
An event on signal b activates both the nand_gate and the xor_gate statements
An event on signal c activates the xor_gate statement

Defining Additional Signals

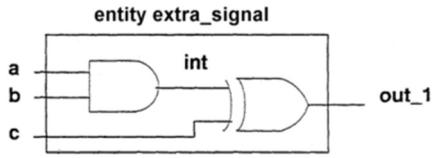
Signals may be declared within declarative area of architecture

- **Format:**

  SIGNAL signal_name : TYPE ;

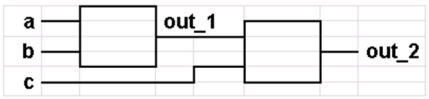
  Example:

```
ARCHITECTURE example OF
extra_signal IS
  SIGNAL int : std_ulogic ;
BEGIN
  int   <= a AND b ;
  out_1 <= c XOR int ;
END example ;
```



Concurrent Signal Change

- **Signals that change during concurrent execution are "queued" to change in the future**
- **Signal assignment statements without an explicit after clause are given a delay of one delta**
- **Example:**



Delta Time

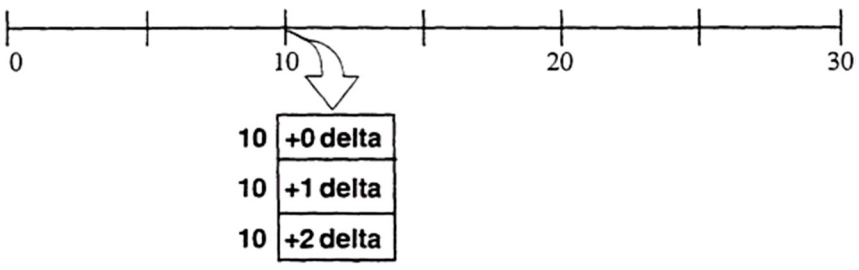
- **A unit of time used in concurrent evaluations**
- **An infinite number of delta times add up to zero simulation time units**
- **All activated statements are evaluated and the results assigned one delta time later**

Delta Time

- **Example:**

```
out_1 <= a NAND b;
```

Time	Inputs		Output
	a	b	out_1
0	0	1	1
10 + 0 delta	1	1	1
10 + 1 delta	1	1	0



Iterations

- **Subsequent activation of statements caused by signal changes produced during execution of previously activated statements**
- **Example:**

```
ARCHITECTURE example OF iterations IS
  SIGNAL out_1 : std_ulogic ;
BEGIN
  out_1 <= a NAND b ;
  out_2 <= out_1 XOR c ;
END example ;
```

Time	Inputs			Internal	Output
	a	b	c	out_1	out_2
0	0	1	1	1	0
1 + 0 delta	1	1	1	1	0
1 + 1 delta	1	1	1	0	0
1 + 2 delta	1	1	1	0	1

Iterations

Given the initial state and subsequent signal change, how many iterations occur in each of the following examples ?

-
```
ARCHITECTURE question_a
OF iterations IS
 BEGIN
   c <= a AND out_2 ;
   out_2 <= b or c ;
 END question_a ;
```

  Current state:
  a b c out_2
  1 0 0 0

  Signal b now changes to 1.
  How many iterations occur?

-
```
ARCHITECTURE question_b OF
iterations IS
  BEGIN
    out_1 <= a XOR out_2 ;
    out_2 <= NOT ( b XOR
out_1) ;
  END question_b ;
```

  Current state:
  a b out_1 out_2
  0 1 0 0

  Signal a now changes to 1.
  How many iterations occur?

Observing Iterations

- **Signal changes may produce successive iterations**
- **Debugger commands are used to single step through each line of code and each iteration**

```
 1 ENTITY empty IS
 2   PORT ( a, b : IN BIT ;
 3          C : OUT BIT ) ;
 4 END empty;
 5
 6
 7 ARCHITECTURE blank OF empty IS
 8
 9 BEGIN
10   - <= ----- ;
11   - <= ----- ;
⇒12  - <= ----- ;
13   - <= ----- ;
14   - <= ----- ;
15 END blank ;
16
17
18
```


Observing Iterations

- **Complete the following table for this example:**

```
ARCHITECTURE question1 OF iterations IS
  SIGNAL int : std_ulogic ;
BEGIN
  int   <= a AND b ;
  out_1 <= c XOR int ;
  out_2 <= a NAND c;
END question 1;
```

Time	Inputs			Internal	Outputs	
	a	b	c	int	out_1	out_2
0	0	1	1	0	1	1
1	1	1	1			
1 + 1 delta	1	1	1			
1 + 2 delta	1	1	1			
2	1	1	1			

Styles of Modelling

- **Structural**

  Hierarchical arrangement of interconnected components

- **Behavioural**

  Functional representation of action and timing

  – **Data Flow**

    Behaviour is shown as a series of register assignments

  – **Algorithmic**

    Behaviour is described in terms of programmatic transformations

Algorithmic nand_gate Model

```
• LIBRARY ieee ;
  USE ieee.std_logic_1164.ALL

• ENTITY nand_gate IS
    PORT ( a, b:  IN std_ulogic ;
              c: OUT std_ulogic );
  END nand_gate ;

• ARCHITECTURE algorithm OF nand_gate IS

  BEGIN
    PROCESS
    BEGIN
      IF a = '1' AND b = '1' THEN
        c <= '0' AFTER 5 ns ;
      ELSE
        c <= '1' AFTER 10 ns ;
      END IF ;
      WAIT ON a, b ;
    END PROCESS ;
  END algorithm ;
```

Process Statement

Concurrent statement that delineates a set of sequentially executed statements

Format:

```
[ label : ]
[ POSTPONED ] PROCESS (sensitivity_list) [IS]--*
   -- declarative_statements
BEGIN
   -- sequential_activity_statements
END [POSTPONED] PROCESS [ label] ;
```

```
Example:
     invert1:
     PROCESS
     BEGIN
       temp_store <= a AND b ;
       c <= NOT temp_store ;
       WAIT ON a, b ;
     END PROCESS invert1 ;

   *The reserved words IS and POSTPONED are IEEE Std 1076-1993
   additions.
```


Labels

- **Provide internal documentation**
- **User may supply labels for:**
  - **Concurrent Assertion Statements**
  - **Concurrent Procedure Calls**
  - **Concurrent Signal Assignments**
  - **Process Statements**
  - **Loop Statements**
  - **Generate Statements**
- **User must supply labels for:**
  - **Block Statements**
  - **Component Instantiation Statements**
- **Example:**

```
 invert1:
  PROCESS
  BEGIN
    a <= NOT b ;
    WAIT ON b ;
  END PROCESS invert1 ;
```

Sequential Operations

- **Statements within processes are executed in the order in which they are written**
- **Example:**

```
ARCHITECTURE example OF proc IS

 BEGIN
   out_1 <= c XOR b ;
   pro_exam: PROCESS
     VARIABLE eo1 : std_ulogic ;
   BEGIN
     eo1 := a AND b ;
     out_2 <= NOT eo1 ;
     WAIT ON a, b ;
   END PROCESS pro_exam ;
 END example ;
```


Sequential Statements

- **Wait Statement**
- **Variable Assignment**
- **Signal Assignment***
- **If Statement**
- **Case Statement**
- **Loops**
- **Next Statement**
- **Exit Statement**
- **Return Statement**
- **Null Statement**
- **Procedure Call***
- **Assertion Statement***

***Have both a sequential and concurrent form**

Wait Statement

- **Provides execution control of process statements**
- **Without control, statements within process execute continually**
- **Format:**
  WAIT [ON list] [UNTIL condition][FOR time] ;
- **Example:**

```
PROCESS
   VARIABLE en : std_ulogic ;
 BEGIN
   en := a AND b ;
   c <= NOT en ;
   WAIT ON a, b ;
 END PROCESS ;
```


Wait Control

- **Suspension may be based on:**
  - **Signal sensitivity**
  - **Condition**
  - **Timeout**
- **More than one suspension criteria may be included in wait statement**
- **Format:**

  WAIT [ON list] [UNTIL condition] [FOR time] ;

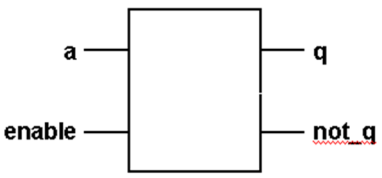
Wait Control

- **Example:**

```
WAIT ;
WAIT ON a ;
WAIT ON b FOR 100 ns ;
WAIT UNTIL a = '1' FOR 50 ns ;
WAIT ON a UNTIL (c = '1') FOR 50 ns ;

WAIT UNTIL (c = '1') ;  -- is equivalent to
WAIT ON c UNTIL (c = '1') ;
```


Gated d_latch

```
ARCHITECTURE example OF d_latch IS

   BEGIN
     latch: PROCESS
     BEGIN
       WAIT ON a UNTIL enable = '1' ;
           q <= a ;
       not_q <= NOT a ;
     END PROCESS latch ;
   END example ;
```



Process Sensitivity List

- Equivalent to a "wait on" statement
- If used, wait statements may not be used in process
- Format:
  [label :] PROCESS (sensitivity_list)
  BEGIN
  --sequential_statements
  END PROCESS [label];
- Example:

```
PROCESS (a, b)
BEGIN
  c <= a AND b ;
END PROCESS ;
```

(Equivalent)

```
PROCESS
BEGIN
  c <= a AND b ;
  WAIT ON a, b ;
END PROCESS ;
```


Variable Declarations

Used only within sequential areas

Format:

VARIABLE var_name : type [:= initial_value] ;

Variable assume value instantly (no time dimension)

Assignment operator:

:=

Example:

```
example : PROCESS
   VARIABLE tla : std_ulogic := '1';
BEGIN
   tla := a AND b ;
   out_2 <= tla XOR x ;
   WAIT on a, b ;
END PROCESS example ;
```

Signal Versus Variable Assignment

```
var_example: PROCESS
  VARIABLE num, sum : integer := 0 ;
BEGIN
  WAIT FOR 10 ns ;
  num := num + 1 ;
  sum := sum + num ;
END PROCESS var_example;
```

```
-----
  SIGNAL num, sum : integer := 0 ;
----
  sig_example : PROCESS
  BEGIN
    WAIT FOR 10 ns ;
    num <= num + 1 ;
    sum <= sum + num ;
  END PROCESS sig_example ;
-----
```


Sequential Signal Assignment

- **Without a transport phrase, only the last value assigned to signal within sequential signal assignment is recognized**
- **Example:**

```
no_transport : PROCESS
BEGIN
  WAIT on c;
  out_1 <= '1' AFTER 10 ns ;
  out_1 <= '0' AFTER 20 ns ;
END PROCESS ;
```

```
with_transport : PROCESS
BEGIN
  WAIT ON c ;
  out_1 <= '1' AFTER 10 ns ;
  out_1 <= TRANSPORT '0' AFTER 20 ns ;
END PROCESS ;
```

Review of Sections 1- 4

- **Model Structure (Entity and Architecture)**
- **Objects (Classes and Types)**
- **Signal and Signal Assignment**
- **Variable, Shared Variables and Variable Assignments**
- **Logical Operators**
- **Delta, Inertial, and Transport Delay**
- **Concurrent and Sequential Operations**
- **Entering and Testing Models**

Sequential Modeling

```
LIBRARY ieee ;
   USE ieee.std_logic_1164.ALL ;

ENTITY nand_gate IS
     PORT ( a, b : IN  bit ;
            c : OUT bit );
   END nand_gate ;

ARCHITECTURE algorithm OF nand_gate IS

BEGIN
     PROCESS (a, b)
     BEGIN
       IF a = '1' AND b = '1' THEN
         c <= '0' AFTER 10 ns ;
       ELSE
         c <= '1' AFTER 10 ns ;
       END IF ;
     END PROCESS ;
   END algorithm ;
```

Controlling Sequential Code

- **Conditional Control:**
  - **If statements**
  - **Case statements**
- **Iterative Control**
  - **Loops**

IF Statement

- **Provides conditional control of sequential statements**
- **Condition in statement must evaluate to a Boolean value**
- **Execution of statements within if statement occurs when condition is "true"**
- **Terminates with END IF**
- **Format:**
  IF condition THEN
  --sequential_statements
  END IF ;
- **Examples:**

```
IF a = '1' THEN
  count := count + 1 ;
END IF ;
```

Relational Operators

- **Return Boolean values as their result**

Operator	Operation
=	equal
/=	not equal
<	less than
<=	less than or equal
>	greater than
>=	greater than or equal

If-Else Construct

- **Provides selection control between two groups of statements based on a single condition**
- **Format:**

```
IF condition THEN
  --sequential_statements
ELSE
  --sequential_statements
END IF ;
```

- **Examples:**

```
IF a = '1' THEN
    ones_cnt := ones_cnt + 1 ;
ELSE
    others_cnt := others_cnt + 1 ;
END IF ;
```

Elsif Construction

- **Provides multiple alternatives based upon multiple conditions**
- **Format:**

```
IF condition THEN
  --sequential_statements
ELSIF condition THEN
  --sequential_statements
END IF ;
```

- **Examples:**

```
IF a = '1' THEN
   ones_cnt := ones_cnt + 1 ;
ELSIF a = '0' THEN
   zeroes_cnt := zeroes_cnt + 1 ;
ELSE
   others_cnt := others_cnt + 1 ;
END IF ;
```

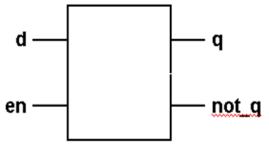

Example: nor_gate

```
LIBRARY ieee ;
   USE ieee.std_logic_1164.ALL ;

ENTITY nor_gate IS
     PORT ( a, b : IN  std_ulogic ;
            c : OUT std_ulogic );
   END nor_gate ;

ARCHITECTURE algorithm OF nor_gate IS
   BEGIN
    nor_process:
    PROCESS (a, b)
    BEGIN
     IF a = '0' AND b = '0' THEN
       c <= '1' AFTER 25 ns ;
     ELSIF (a = '1') OR (b ='1') THEN
       c <= '0' AFTER 25 ns ;
     ELSE
       c <= 'X' AFTER 10 ns ;
     END IF ;
    END PROCESS nor_process ;
   END algorithm ;
```

Modeling Enables

- **Enables active devices or puts the devices in operating mode**
- **Examples:**

```
ARCHITECTURE algorithm OF
gated_d_latch IS
BEGIN
  d_latch :
  PROCESS (en, d)
  BEGIN
    IF en = '1' THEN
      q <= d AFTER 25 ns ;
      not_q <= NOT d AFTER 25 ns ;
    END IF ;
  END PROCESS d_latch ;
END algorithm ;
```



Modeling Oscillators

- **Simple oscillators switch back and forth between two states**
- **Example:**

```
ARCHITECTURE algorithm OF oscillator IS
  SIGNAL clk : std_ulogic := '0' ;
BEGIN
  clock : PROCESS
    VARIABLE switch : BIT := '0' ;
  BEGIN
    IF switch = '0' THEN
      clk <= '0' ;
    ELSE
      clk <= '1' ;
    END IF ;
    switch := NOT switch ;
    WAIT FOR 25 ns ;
  END PROCESS clock ;
END algorithm ;
```

Modeling Clocks

```
clock_1 : PROCESS

BEGIN
        clk <= NOT clk ;
        WAIT FOR 25 ns ;

END PROCESS clock_1 ;
```

```
clock_3 : PROCESS (clk)

BEGIN
        clk <= NOT clk AFTER 25 ns ;

END PROCESS clock_3 ;
```

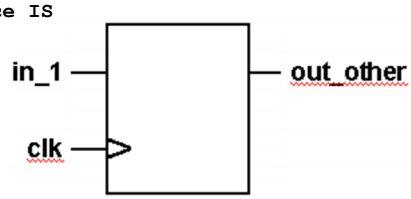
```
clock_2 : PROCESS

BEGIN
        clk <= NOT clk AFTER 25 ns ;
        WAIT ON clk ;

END PROCESS clock_2 ;
```

```
clk <= NOT clk AFTER 25 ns
```


Modeling Synchronous Devices

- Changes in output occur on a triggering input
- Example:

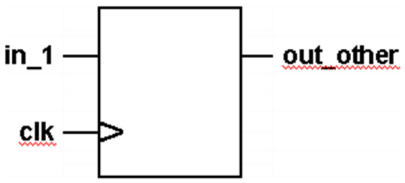
```
ARCHITECTURE example OF sync_device IS
   BEGIN
     in_out : PROCESS (clk)
     BEGIN
       out_other <= in_1 ;
     END PROCESS in_out ;
   END example ;
```



Modeling Synchronous Devices

- Changes occur only at a specific point on a triggering input
- Typically occurs on rising or falling edge of clock pulse
- Example:

```
ARCHITECTURE example OF rising_edge IS
BEGIN
  in_out : PROCESS (clk)
  BEGIN
    IF clk = '1' THEN
      out_other <= in_1 ;
    END IF ;
  END PROCESS in_out ;
END example ;
```



Attributes

- Specific values associated with given objects
- Certain attributes for signals, arrays, and types are predefined
- Additional attributes may be defined by the user
- Format:
  object_name'attribute_designator
- Example:
  `clock'ACTIVE`

Predefined Signal Attributes

- **signal'EVENT – returns value "true" or "false" if event occured in present delta time period**
- **signal'ACTIVE – returns value "true" or "false" if activity occurred in present delta time period**
- **signal'STABLE(t) – returns a signal value "true" or "false" based on event in (t) time units**
- **signal'QUIET(t) – returns a signal value "true" or "false" based on activity in (t) time units**
- **signal'TRANSACTION – returns an event whenever there is activity on the signal**
- **signal'DELAYED(t) – returns a signal delayed (t) time units**
- **signal'LAST_EVENT – returns amount of time since last event**
- **signal'LAST_ACTIVE – returns amount of time since last activity**
- **signal'LAST_VALUE – returns value equal to previous value**

Checks With Attributes

- **Rising Clock Edge**

```
clk'EVENT AND clk = '1' AND clk'LAST_VALUE = '0'
```

or for bit type only

```
NOT clk'STABLE and clk = '1'
```

- **Falling Clock Edge**

```
clk'EVENT AND clk = '0' AND clk'LAST_VALUE = '1'
```

or for bit type only

```
NOT clk'STABLE and clk = '0'
```

- **Pulse Width**

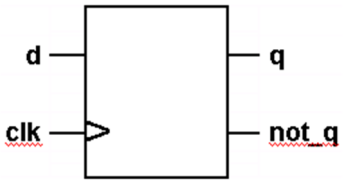
```
signal'LAST_EVENT >= 10 ns
```

- **Stability**

```
signal'STABLE(10 ns)
```

Edge Triggered d_flip_flop

```
ARCHITECTURE example OF d_flip_flop IS
  BEGIN
    d_flop : PROCESS (clk)
    BEGIN
      IF clk'EVENT AND clk = '1' THEN
        q <= d ;
        not_q <= NOT d ;
      END IF ;
    END PROCESS d_flop ;
  END example ;
```



Remark: clk'EVENT is redundant since process is sensitive.

Checking Setup Time

- **Minimum interval for level prior to triggering edge for reliable entry**

- **Example:**

```
ARCHITECTURE example OF setup_check IS
constant min_setup_time: TIME := 5 ns ;
BEGIN
  d_flop : PROCESS (clk)
  BEGIN
    IF clk'LAST_VALUE = '0' AND clk = '1' AND
     d'STABLE (min_setup_time) THEN
      q <= d ;
      not_q <= NOT d ;
    END IF ;
  END PROCESS d_flop ;
END example ;
```

Checking Hold Time

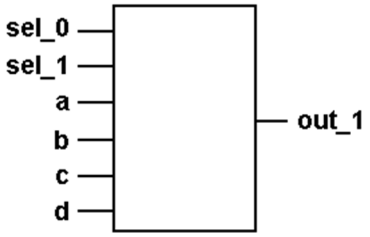
- **Minimum interval for level prior to triggering edge for reliable entry**

- **Example:**

```
ARCHITECTURE example OF hold_check IS
BEGIN
  PROCESS
  BEGIN
    WAIT UNTIL clk = '1' AND clk'LAST_VALUE = '0';
    WAIT FOR min_hold_time ;
    IF d'STABLE (setup_and_hold_time) THEN
      q <= d ;
    ELSE
      q <= 'X' ;
    END IF ;
  END PROCESS ;
END example ;
```


Data Selection

```
 1 ARCHITECTURE example OF multiplexer IS
 2 BEGIN
 3   one_of_four: PROCESS (sel_0, sel_1, a, b, c, d)
 4   BEGIN
 5     IF sel_0 = '0' THEN
 6       IF sel_1 = '0' THEN
 7         out_1 <= a ;
 8       ELSE
 9         out_1 <= b ;
10       END IF ;
11     ELSIF sel_0 = '1' THEN
12       IF sel_1 = '0' THEN
13         out_1 <= c ;
14       ELSE
15         out_1 <= d ;
16       END IF
17     END IF
18   END PROCESS one_of_four ;
19 END example ;
```



Conditional Signal Assignment

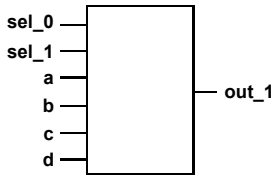
- **Form of concurrent signal assignment statement**
- **Equivalent to signal assignment using if statements**
- **Condition must evaluate to Boolean**
- **Last "else" must not be followed by "when"**
- **Format:**

```
target <= waveform WHEN condition ELSE
                  waveform ;
```

- **Example:**

```
a_target <= '1' WHEN a_source = '0' ELSE
            'X' ;
```


Using Conditional Assignment




```
1 ARCHITECTURE example OF multiplexer IS
2 BEGIN
3
4   out_1 <= a WHEN sel_0 = '0' AND sel_1 = '0' ELSE
5            b WHEN sel_0 = '0' AND sel_1 = '1' ELSE
6            c WHEN sel_0 = '1' AND sel_1 = '0' ELSE
7            d WHEN sel_0 = '1' AND sel_1 = '1' ELSE
8            'X' ;
9 END example ;
```

Selected Signal Assignment

- **Selection is based on value of expression**
- **Format:**

```
WITH expression SELECT
  target <= waveform WHEN expression_value
            waveform WHEN expression_value ;
```

- **Example:**

```
WITH a_example SELECT
  a_target <= waveform_1 WHEN '0' ,
              waveform_2 WHEN '1' ;
```


When Clause

- **Required for selected signal assignment statements**
- **No two when clauses may have the same value**
- **All possible values of "expression" must be accounted for**
- **Format:**

```
WITH expression SELECT
  target <= waveform WHEN expression_value ;
```

- **Example:**

```
WITH a_example SELECT
     a_target <=   waveform_1 WHEN '0' ,
                   waveform_2 WHEN '1' ;
```

Others

- **Indicates all unspecified elements**
- **If used, must be last item in list**
- **Example:**

```
WITH a_example SELECT
     a_target <=   waveform_1 WHEN '0' ,
                   waveform_2 WHEN '1' ,
                   a_target WHEN OTHERS;
```

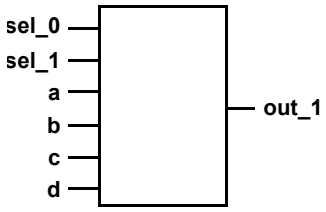

Concatenation

- **Linking two or more items into a single unit**
- **Ampersand (&) is used to indicate concatenation**
- **Items being concatenated must be of same type**
- **Format:**
  left_operand & right_operand
- **Example:**

```
signal_0 <= '0' ;
signal_1 <= '1' ;
total <= signal_0 & signal_1 ; -- results in "01"
```

Concurrent one_of_four

```
ARCHITECTURE a1 OF mux4 IS
  BEGIN

  WITH (sel_0 & sel_1) SELECT
    out_1 <=  a WHEN “00” ,
              b WHEN “01” ,
              c WHEN “10” ,
              d WHEN “11” ,
              ’X’ WHEN OTHERS ;
END a1 ;
```



Sequential Selection

- When-Else and With-When structures cannot be used in sequentially processed model areas (e.g. VHDL-Processes)
- In sequential areas, if statements and case statements are used to select statements for execution
- Example of case statement:

```
CASE in_put IS
    WHEN ’1’ => out_put <= ’0’ ;
    WHEN ’0’ => out_put <= ’1’ ;
  END CASE ;
```

Case Statement

- Controls execution of one or more sequential statements
- Format:

```
CASE expression IS
  WHEN exprn_value => seq_statements ;
END CASE ;
```

- Example:

```
ARCHITECTURE example OF case_statement IS
BEGIN
  in_vert : PROCESS (in_put)
  BEGIN
    CASE in_put IS
     WHEN '1' => out_put <= '0' ;
     WHEN '0' => out_put <= '1' ;
                 def_con <= 'X' ;
     WHEN OTHERS => out_put <= 'X' ;
    END CASE;
  END PROCESS in_vert ;
END example ;
```

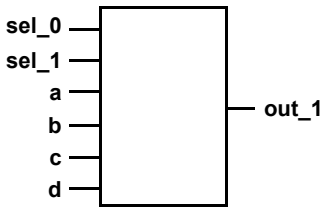

Null

- Sequential statement indicating no action occurs
- An action is required for all "expression" values in a case statement
- Used for "expression" values that require no action
- Example:

```
CASE opcode IS
  WHEN '0' => out_1 <= in_1;
  WHEN '1' => out_1 <= in_0;
  WHEN OTHERS => NULL ;
END CASE ;
```

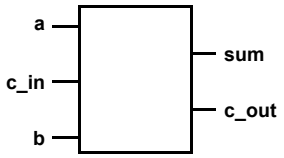
Sequential one_of_four

```
1 ARCHITECTURE a2 OF case_mux IS
2  BEGIN
4   one_of_four: PROCESS (sel_0, sel_1, a, b, c, d)
5   BEGIN
6     CASE (sel_0 & sel_1) IS
7       WHEN “00” => out_1 <= a ;
8       WHEN “01” => out_1 <= b ;
9       WHEN “10” => out_1 <= c ;
10       WHEN “11” => out_1 <= d ;
11       WHEN OTHERS => NULL ;
12     END CASE ;
13   END PROCESS one_of_four ;
14 END a2 ;
```



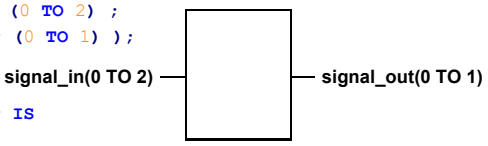
one_bit_adder

```
library ieee;
use ieee.std_logic_1164.all;
entity one_bit_adder is
port(     a, c_in, b: in bit;
          sum, c_out: out std_ulogic);
end one_bit_adder;
ARCHITECTURE behav OF one_bit_adder IS
 SIGNAL total : integer RANGE 0 TO 4 ;
 BEGIN
  majority: WITH (a & c_in & b) SELECT
  total <=0 WHEN "000",
          1 WHEN "100" | "010" | "001" ,
          2 WHEN "101" | "011" | "110" ,
          3 WHEN "111" ,
          4 WHEN OTHERS ;
    adder : PROCESS (total)
   BEGIN
     CASE total IS
          WHEN 0 => sum <= '0'; c_out <= '0' ;
          WHEN 1 => sum <= '1'; c_out <= '0' ;
          WHEN 2 => sum <= '0'; c_out <= '1' ;
          WHEN 3 => sum <= '1'; c_out <= '1' ;
          WHEN OTHERS => sum <= 'X' ; c_out <= 'X';
     END CASE;
   END PROCESS;
 END behav;
```



Array Input one_bit_adder

```
ENTITY adder IS
PORT (   signal_in :  IN bit_vector (0 TO 2) ;
        signal_out : OUT bit_vector (0 TO 1) );
END adder ;

ARCHITECTURE array_example OF adder IS
  BEGIN
   add_up : PROCESS (signal_in)
        BEGIN
     CASE signal_in IS
     WHEN "000"  => signal_out <= "00";
     WHEN "001" |"010"| "100" => signal_out <="10";
     WHEN "011" |"101"| "110" => signal_out <= "01";
          WHEN "111"  => signal_out <= "11";
     END CASE ;
   END PROCESS add_up ;
 END array_example ;
```



Arrays

- **Collections of values all belonging to the same object**
- **Individual values stored within separate elements**
- **Individual elements identified by an index value**
- **Set up requires an array type declaration and an object declaration of that type**
- **Constrained and unconstrained array declarations, but only constrained objects**
- **Type Declarations Format:**

  TYPE name IS ARRAY (index_range) OF element_type ;

- **Examples:**

```
TYPE array_type IS ARRAY (0 TO 10) OF bit;
TYPE bit_vector IS ARRAY (natural RANGE <>) OF bit;
SIGNAL sig1 : array_type ;
SIGNAL sig2 : bit_vector(0 TO 3);
```

Predefined Array Attributes

- **Return information about index values**
- **Formats:**
  array_name' LEFT array_name'HIGH
  array_name'RIGHT array_name'LOW
  array_name'RANGE array_name'LENGTH
  array_name'REVERSE_RANGE
- **Examples:**

```
SIGNAL ray : bit_vector(0 TO 7) ;

ray'LEFT            --returns 0
ray'RIGHT           --returns 7
ray'HIGH            --returns 7
ray'LOW             --returns 0
ray'RANGE           --returns 0 TO 7
ray'LENGTH          --returns 8
ray'REVERSE_RANGE   --returns 7 DOWNTO 0
```


Loops

- **Sequences of statements that are executed repeatedly**
- **Types of loops:**
  - **For**
  - **While**
  - **Loop – exit construct**
- **Format:**

```
[loop_label : ]
iteration_scheme
LOOP
  sequence_of_statements ;
END LOOP [loop_label] ;
```

For Loop

- **Statements are executed once for each value in loop parameter's range**
- **Loop parameter is implicitly declared and may not be modified within loop or used outside loop**
- **Format:**
  [label :] FOR loop_parameter IN discrete_range
  LOOP
  --sequential_statements
  END LOOP [label] ;
- **Example:**

```
PROCESS (signal_a)
BEGIN
  lbl: FOR index IN 0 TO 7
  LOOP
    ray_out(index) <= ray_in(index) ;
  END LOOP lbl ;
END PROCESS ;
```


While Loop

- **Execution of statements within loop is controlled by Boolean condition**
- **Condition is evaluated before each repetition of loop**
- **Format:**
  WHILE boolean_expression
  LOOP
  --sequential_statements
  END LOOP ;
- **Example:**

```
p1 : PROCESS (signal_a)
  VARIABLE index : integer := 0 ;
BEGIN
  from_in_to_out :
  WHILE index < 8 LOOP
    ray_out(index) <= ray_in(index) ;
    index := index + 1 ;
  END LOOP from_in_to_out ;
END PROCESS p1 ;
```

Loop - Exit Construct

- **Does not contain an iteration scheme specifying how loop terminates**
- **A decision to leave loop must be made from within loop**
- **Can be terminated from within by:**
  - **EXIT**
  - **NEXT**
- **Format:**

```
LOOP
  --sequential_statements
  --including decision_to_quit
END LOOP ;
```


Exit Statement

- **Used to break out of loops**
- **Only occurs within loops**
- **Format:**

```
EXIT;
EXIT loop_label ;
EXIT WHEN boolean_expression ;
EXIT loop_label WHEN boolean_expression ;
```

Exit Statement

- **Example:**

```
show_loops : PROCESS (s)
  VARIABLE sum, cnt : integer := 0 ;
BEGIN
  sum := 0; cnt := 0;
  first : LOOP
    cnt := cnt + 1 ;
    sum := sum + cnt ;
    EXIT WHEN sum > 100 ;
  END LOOP first ;
  second: LOOP
    cnt := cnt + 1 ;
    sum := sum + cnt ;
    IF cnt > 100 THEN EXIT;
    END IF ;
  END LOOP second ;
END PROCESS show_loops ;
```

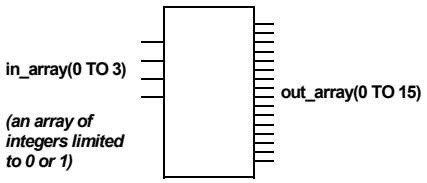

Next Statement

- **Causes execution to jump to the beginning of the loop and start the next iteration**
- **Format:**
  NEXT ;
  NEXT loop_label ;
  NEXT WHEN boolean_expression ;
  NEXT loop_label WHEN boolean_expression ;
- **Example:**

```
nxt_xmp1 : PROCESS (s)
  VARIABLE temp, j : integer := 0 ;
BEGIN
  temp := 0; j := 0 ;
  a_loop : FOR I IN 0 TO 7 LOOP
    j := j + 1 ;
    IF j > 5 THEN NEXT a_loop ;
    END IF ;
    temp := temp + 1 ;
  END LOOP a_loop ;
END PROCESS nxt_xmpl ;
```

four_to_sixteen Decoder

```
 1 four_to_sixteen :
 2 PROCESS (in_array)
 3   VARIABLE sum : integer RANGE 0 TO 15 ;
 4 BEGIN
 5   sum := 0;
 6   sum_input:
 7    FOR I IN 0 TO 3 LOOP
 8      sum := sum + (2**i) * in_array(i);
 9    END LOOP sum_input;
10   zero_array :
11    FOR j IN out_array'RANGE LOOP
12      IF (j = sum) THEN
13        out_array(j) <= '1' ;
14      ELSE
15        out_array(j) <= '0' ;
16      END IF ;
17    END LOOP zero_array ;
18 END PROCESS four_to_sixteen;
```



Assert Statement

- **Checks condition and reports message with severity level if condition is <u>not</u> true**
- **Format:**
  ASSERT condition ;
  ASSERT condition REPORT "message" ;
  ASSERT condition SEVERITY level ;
  ASSERT condition REPORT "message" SEVERITY level ;
- **Example:**

```
ASSERT signal_in = '1'
  REPORT "Input is not 1"
    SEVERITY Warning ;
```

Severity Levels

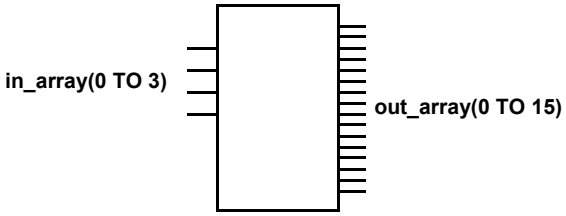
- **Levels of severity from least to greatest:**
  - **Note – general information**
  - **Warning – undesirable condition**
  - **Error – task completed, result wrong**
  - **Failure – task not completed**
- **Simulation stops when severity matches or exceeds specified severity level**
- **Simulators generally default to severity of "failure"**

Placing Assert Statements

- **May appear within:**
  - **concurrent statement areas**
  - **sequential statement areas**
  - **statement area of entity declaration**
- **Example:**

```
 1 ENTITY rs_flip_flop IS
 2   PORT (r, s     :  IN bit ;
 3         q, not_q : OUT bit );
 4 END rs_flip_flop ;
 5
 6 ARCHITECTURE behave OF rs_flip_flop IS
 7 BEGIN
 8   ASSERT NOT ( r = '1' AND s = '1')
 9    REPORT "race condition"
10     SEVERITY FAILURE ;
      .
      .
      .
xx END behave ;
```

Asserted Decoder



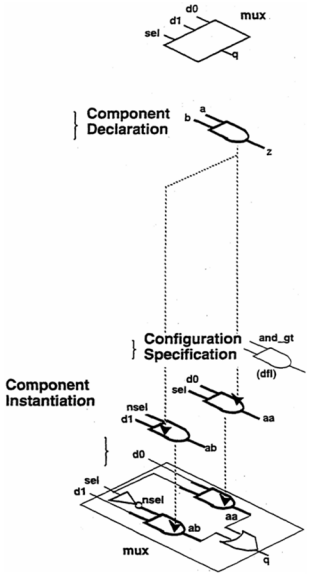

```
 1 four_to_sixteen_decoder :
 2 PROCESS (in_array)
 3   VARIABLE sum : integer RANGE 0 TO 15 ;
 4 BEGIN
 5   ASSERT NOT (in_array(0) = 'X' OR
 6               in_array(1) = 'X' OR
 7               in_array(2) = 'X' OR
 8               in_array(3) = 'X')
 9    REPORT "unknown value"
10     SEVERITY ERROR;
11   sum := 0;
         . . .
xx END PROCESS;
```


Structural Modeling

- **Describes entity in terms of its subcomponents and how they are connected**
- **Hierarchical arrangement of interconnected components**
- **Ports and their interconnection are central focus**
- **Much like netlisting or like a schematic with instances**
- **Has to be supported at some level by behavioural description (i.e. independent VHDL-models)**

Structural Steps

1. Component Declaration
2. Component Specification
3. Component Instantiation



Component Declaration

- **Describes local interface**
  - **Name**
  - **Ports and generics**
- **Appears in declarative areas**
- **Format:**

```
COMPONENT component_name
   [GENERIC (local_generic_list) ; ]
   [ PORT (local_port_list) ; ]
END COMPONENT ;
```

- **Example:**

```
COMPONENT and_gate
   GENERIC ( delay : time ;
             load : integer );
   PORT (a, b:  IN std_ulogic ;
            c: OUT std_ulogic );
END COMPONENT ;
```

Component Specification

- **Specifies which design entity is to be used as component**
- **States:**
  - Name of entity
  - Name of architecture
- **Format:**

  FOR comp_instan_label : component_name
  USE ENTITY library.entity_name[(architecture_name)] ;

- **Example:**

```
FOR us2 : and_gate
 USE ENTITY work.my_component(arch_one);
```


Component Instantiation

- **Places component in a relationship to other components**
- **Identifies signal connected to component ports**
- **Associates values to generics**
- **Format:**

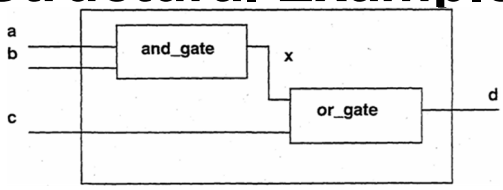
  *** label : component_name**
  [GENERIC MAP (association_list)]
  [PORT MAP (association_list)];

- **Example:**

```
u2 : and_gate
   GENERIC MAP (delay => 2.5 ns, load => 10)
   PORT MAP (c => signal1, a => p1, b => p2 );
```

- *** label is required**

Structural Example



- **Specification**

  Create model of entity with three input ports (a, b, c) and one output port (d). Signals a and b are ANDed and the result (x) is ORed with signal c to produce output signal d.

- **Analysis:**

  Two components will be declared, specified, and instantiated. One for the and_gate and the other for the or_gate. In addition, an internal signal x, which represents the output of the and_gate and the input of the or_gate, will be defined.

Structural Model

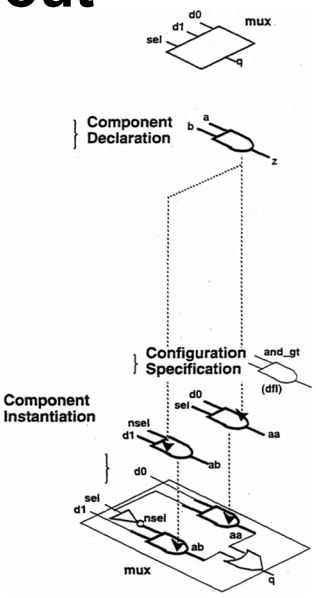
```
 1
 2 LIBRARY ieee ;
 3 USE ieee.std_logic_1164.ALL ;
 4
 5 ENTITY my_thing IS
 6   PORT ( a, b, c :  IN std_ulogic ;
 7                d : OUT std_ulogic) ;
 8 END my_thing ;
 9
10
11 ARCHITECTURE struct OF my_thing IS
12   COMPONENT and1
13     PORT (x, y :  IN std_ulogic ;
14                z : OUT std_ulogic );
15     END COMPONENT ;
16
17   COMPONENT or1
18     PORT (x, y :  IN std_ulogic ;
19                z : OUT std_ulogic );
20     END COMPONENT ;
21
22   FOR u1:and1
23     USE ENTITY work.and_gate(beh);
24
25   FOR u2:or1
26     USE ENTITY work.or_gate(beh);
27
28   SIGNAL x : std_ulogic ;
29
30 BEGIN
31   u1 : and1 PORT MAP (a, b, x) ;-- short format of PORT MAP (x=>a,..)
32   u2 :  or1 PORT MAP (x, c, d) ;
33 END struct ;
```

Structural Layout

```
ENTITY mux IS
  PORT (d0, d1, sel:  IN std_ulogic;
        q: OUT std_ulogic);
END mux ;
```

```
ARCHITECTURE struct OF mux IS
  COMPONENT and2
    PORT (a, b: IN  std_ulogic ;
          z: OUT std_ulogic);
  END COMPONENT ;

  COMPONENT or2
    PORT (a, b: IN  std_ulogic ;
          z: OUT std_ulogic);
  END COMPONENT ;

  COMPONENT inv
    PORT (i: IN  std_ulogic ;
          o: OUT std_ulogic);
  END COMPONENT ;

  SIGNAL aa, ab, nsel: std_ulogic ;

  FOR u1 : inv USE ENTITY invrt(behave) ;
  FOR u2,u3: and2 USE ENTITY and_gt(dfl) ;
  FOR u4 : or2 USE ENTITY or_gt(arch1) ;

BEGIN
```

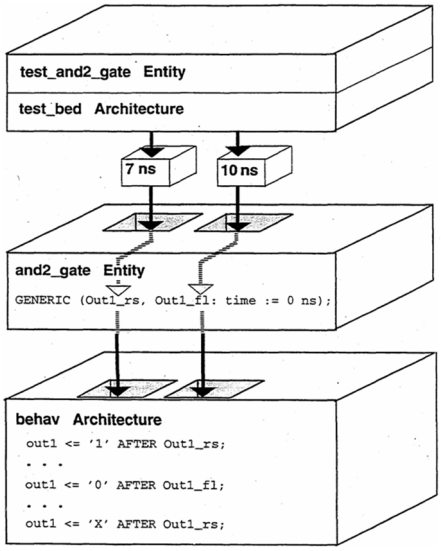
```
  u1: inv  PORT MAP (sel, nsel);
  u2: and2 PORT MAP (nsel, d1, ab);
  u3: and2 PORT MAP (d0, sel, aa);
  u4: or2  PORT MAP (aa, ab, q);

END struct  ;
```



Generics

- **Provides method for passing additional data into model**
- **Declared in interface lists in:**
  - **entity declaration**
  - **component declaration**
  - **blocks**
- **Data is assigned to generics from:**
  - **generic mapping statements**

Generic Use Example



Generic Statement

- **Used to construct parameterized hardware components**
- **Declare generic before port definition within entity**
- **Format:**

```
GENERIC ([class] identifier : [MODE] type [:=default_value]);
```

- **Example:**

```
GENERIC (CONSTANT de_lay : IN time := 5 ns );
  --or
GENERIC (de_lay : time := 5 ns ) ;
```

- **Use:**

```
ENTITY generics_use IS
  GENERIC ( de_lay : time := 5 ns ) ;
  PORT ( in_a :  IN std_ulogic;
         out_b : OUT std_ulogic);
END generic_use ;

ARCHITECTURE example OF generic_use IS
BEGIN
  out_b <= in_a AFTER de_lay ;
END example ;
```

Mapping Generic Values

- Values may be assigned to identifiers in generic statement through generic map
- Format:

  GENERIC MAP (value_1, value_2, …) ;
- Each value in map must be associated with an identifier in generic list
- Generic maps may appear in component instantiation, and block headers
- Example:

```
u2:and_gate
   GENERIC MAP (3 ns)
   PORT MAP (signal1, signal2, signal3);
```


Default Binding

```
Entity fulladder is
Port(   x, y, cin: in bit;
     sum, cout: out bit);
end fulladder;

Architecture struct of fulladder is
signal s1, s2, s3: Bit;
component halfadder
Port (  x, y: in bit;
     sum, cout: out bit);
end component;

Begin
     u1: halfadder Port map (x, y, s1, s2);
     u2: halfadder Port map (s1, cin, sum, s3);
cout <= s2 or s3;
end struct;
```

Default Binding:

System inserts:

```
FOR u1,u2:halfadder USE ENTITY
work.halfadder(behave);
```

Default Binding

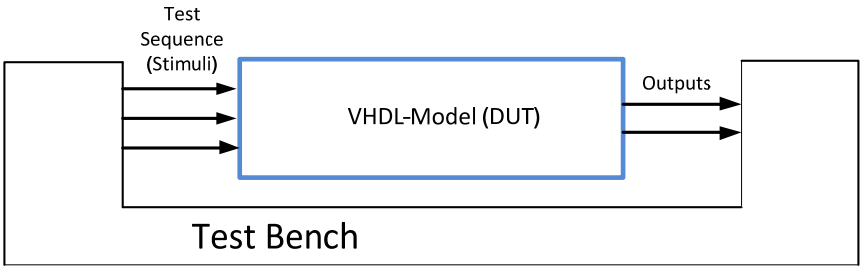
```
Entity Halfadder is
generic (sum_delay   : time:=10 ns;
         carry_delay : time:=5 ns);
port ( x,y :          in bit;
       sum, cout:     out bit);
End Halfadder;

Architecture behave of halfadder is
begin
   sum <= x xor y after sum_delay;
   cout <= x and y after carry_delay;
end behave;
```

Note: The entity to be instantiated has to be compiled first!

Hierarchical designs have to compiled bottom-up!

Test Benches

Used to verify VHDL-models



- **stimulus-only test bench: DUT + test sequence**
- **self-checking test bench: DUT + test sequence + error check**

Stimulus-Only Test Bench

```
-- testbench_for_MUX4.vhd
---------------------------------
entity Testbench is
end Testbench;

architecture TestbenchArch of Testbench is
signal S_in: bit_vector(1 downto 0);
signal E_in: bit_vector(3 downto 0);
signal Y_out: bit;
component MUX4
port(    S: in bit_vector(1 downto 0);
         E: in bit_vector(3 downto 0);
         Y: out bit);
end component;
for u1: MUX4 use entity work.MUX4;
begin
u1: MUX4 port map(S_in, E_in, Y_out);
  E_in <= "1011", "0100" after 400 ns;
  S_in <= "00","01" after 100 ns,"10" after 200 ns,"11" after 300 ns,
"00" after 400 ns,"01" after 500 ns, "10" after 600 ns, "11" after 700 ns;
end architecture TestbenchArch;
```

port map short format:
port map (S_in, E_in, Y_out);

port map long format:
port map (S=>S_in, E=>E_in, Y=>Y_out);

Stimulus-Only Test Bench

DUT: MUX4

```
-- multiplexer.vhd
---------------------------
entity MUX4 is
        port(    S: in bit_vector(1 downto 0);
                 E: in bit_vector(3 downto 0);
                 Y: out bit);
end MUX4;

architecture Behaviour of MUX4 is
begin
        with S select
        Y <=     E(0) when "00",
                 E(1) when "01",
                 E(2) when "10",
                 E(3) when "11";
end architecture Behaviour;
```

Self-Checking Test Bench

```
entity Testbench is
end Testbench;
architecture TestbenchArch of Testbench is
signal S_in: bit_vector(1 downto 0);
signal E_in: bit_vector(3 downto 0);
signal Y_out: bit;
component MUX4
port(    S: in bit_vector(1 downto 0);
         E: in bit_vector(3 downto 0);
         Y: out bit);
end component;
for u1: MUX4 use entity work.MUX4;
begin
u1: MUX4 port map(S_in, E_in, Y_out);
  TESTPATTERN_ERROR_CHECK : process
  begin
  E_in <= "1011";
  S_in <= "00";
  assert (Y_out = '1') report "ERROR : Test failed" severity error;
  wait for 100 ns;
  S_in <= "01";
  wait for 100 ns;
  S_in <= "10";
  wait;
  end process TESTPATTERN_ERROR_CHECK;
end architecture TestbenchArch;
```

With the use of the *assert* statement you can check whether the output is false or true. If the output is false the *report* command will force a text output.

Advanced Topics

- **Design partitioning: subprograms**
  - **Functions, Procedures**
- **Textual input and output**
- **Types**
- **Arithmetic Operations and Type Conversion**
- **Package, Library**

Design Partition

- **Dividing a model into smaller pieces reduces complexity to manageable levels**
- **Method:**

  - Subprograms

Subprograms

- **Sequences of statement that can be executed from multiple locations in a model**
- **Provide method for breaking large segments of code into smaller segments**
- **Types of subprograms:**
  - **Functions:**

    Used to calculate and return one value
  - **Procedures:**

    Used to define an algorithm that affect many values or no values

Functions

- Provide a method of performing complex algorithms
- Produce a single return value
- Invoked by an expression
- Must contain a return clause
- May consist of two parts:
  - Declaration
  - Body

Function Declaration

- Specifies:
  - Function name
  - Input parameters
  - Type of return value
- Format:
  FUNCTION name (parameter_list) RETURN type is
- Example:

```
FUNCTION exam ( SIGNAL signal_a : bit )
 RETURN bit IS
```

Parameter List

- **Identifies class, name, mode, and type of objects that contain values passed to function**
- **Format:**

  (CLASS object_name : [MODE] TYPE)
- **Object class may be SIGNAL or CONSTANT**
- **Mode is limited to “in”**
- **Type of objects must correspond to type of values passed**
- **Example:**

```
FUNCTION exmp1 (SIGNAL signal_a : IN bit) ...
FUNCTION exmp1 (SIGNAL signal_a : bit) ...
*FUNCTION exmp1(signal_a : bit) ...
    *Object class defaults to
     signal if not stated
```


Return Type

- **Identifies type of value returned by function**
- **Type must be declared**
- **Return value must be of same type as identified by “RETURN type”**
- **Format:**

  … RETURN type
- **Example:**

```
... RETURN boolean
... RETURN bit
```

Function Body

- Defines how functions behaves
- Consists of sequential statements
- Returns only one value
- Must contain a return statement
- Format:

```
FUNCTION name (parameter_list)
  RETURN type IS
    --declarative_statements
BEGIN
    --sequential_statements
    --including return statement
END name ;
```


Return Statement

- Returns value to object assigned to receive function output
- Terminates subprogram execution
- Used only in subprograms
- Format:

Functions:
RETURN expression ;
Procedures:
RETURN;

- Example:

```
FUNCTION example (a, b: IN bit)
  RETURN boolean IS
BEGIN
  RETURN a = b ;
END example;
```

Transfer_check Function

```
 1 FUNCTION transfer_check
 2   (SIGNAL a_ray : IN bit_vector(0 TO 7) ;
 3    SIGNAL b_ray : IN bit_vector(0 TO 7) )
 4      RETURN boolean IS
 5    VARIABLE sum1, sum2 : integer := 0 ;
 6 BEGIN
 7    FOR I IN a_ray'RANGE LOOP
 8      IF a_ray(i) = '1' THEN
 9        sum1 := sum1 + 1 ;
10      END IF ;
11      IF b_ray(i) = '1' THEN
12        sum2 := sum2 + 1 ;
13      END IF ;
14    END LOOP ;
15    RETURN sum1 = sum2 ;
16 END transfer_check ;
```


Function Calls

- **Stated as expression**
- **Cause execution of function body**
- **Specify passing parameters**
- **Occur at or below level of function declaration**
- **Format:**

  function_name (passing_parameter_list)
- **Example:**

```
IF transfer_check (array_1, array_2) ...
```

Passing Parameter List

- **Specifies objects from which values pass to function**
- **Positional association:**
  - Parameters in call match order of parameters in function parameter list
  - Example:

```
FUNCTION tran_ck (SIGNAL a_1: IN bit;
                  SIGNAL b_2: IN bit) ...
              .                   .
              .                   .
IF tran_ck (ray_1, ray_2) ...
```

- **Named association:**
  - Relates parameters in call by name to parameters in function list
  - Example:

```
FUNCTION tran_ck (SIGNAL a_1: IN bit;
                  SIGNAL b_2: IN bit) ...
              .                   .
              .                   .
IF tran_ck (b_2 => ray_2, a_1 => ray_1) ...
```


bit_to_boolean

```
 1 ENTITY example IS
 2   PORT (in_a : IN bit ;
 3         out_b: OUT boolean );
 4 END example ;
 5
 6
 7 ARCHITECTURE func OF example IS
 8   FUNCTION bit_to_boolean (bit_in : IN bit)
 9    RETURN boolean IS
10   BEGIN
11     IF bit_in = '1' THEN
12       RETURN true ;
13     ELSE
14       RETURN false ;
15     END IF ;
16   END bit_to_boolean ;
17 BEGIN
18   out_b <= bit_to_boolean (in_a);
19 END func ;
```

Procedures

- Provide a method of performing complex algorithms
- May produce multiple output values
- May affect input parameters
- Are invoked by a statement
- May consist of two parts:
  - Declaration
  - Body
- May contain a return statement

Procedure Declaration

- Specifies:
  - Procedure name
  - Parameters
- Format:

  PROCEDURE name (parameter_list) IS

- Example:

```
PROCEDURE exam ( VARIABLE var_a  :  IN bit ;
                 SIGNAL signal_a : OUT boolean) IS
```

Parameter List

- **Identifies class, name, mode, and type of objects that contain values passed to and from procedure**
- **Mode determines what actions can be taken on parameter**
- **Object class may be:**
  Constant, signal, or variable
- **Mode may be:**
  In, out, or inout
- **Format:**
  … (class object : mode type) …
- **Example:**

```
PROCEDURE a (VARIABLE a : IN real ) ...
  PROCEDURE b (SIGNAL sig_a: bit ) ...
  PROCEDURE c ( sig_a : BIT ) ...
```


Parameter Modes

- **Determine how parameters are accessed with procedures**
- **Modes:**
  - **In – parameter may be read into procedure but may not be modified**
  - **Out – parameter is created in procedure for use by calling code and may not be passed to procedure from calling code**
  - **Inout – parameter may be read from calling code, modified, and returned to calling code**
- **Defaults for class when a mode is given:**
  - **Class defaults to constant for mode "in"**
  - **Class defaults to variable for mode "out" or "inout"**

Procedure Body

- **Defines how procedure behaves**
- **Consists of sequential statements**
- **Placed in declarative statement areas**
- **May read, create, or modify parameters**
- **Format:**

```
PROCEDURE name (parameter_list) IS
  --declarative_statements
BEGIN
  --sequential_statements
END name ;
```


find_minimum Procedure

```
 1 PROCEDURE find_min
 2  (VARIABLE  a_ray :  IN bit_vector;
 3   VARIABLE min_val : INOUT integer ;
 4   VARIABLE old_min:  OUT integer ) IS
 5 BEGIN
 6  old_min := min_val ;
 7  FOR I IN a_ray'RANGE LOOP
 8    IF a_ray(i) < min_val THEN
 9      min_val := a_ray(i) ;
10    END IF;
11  END LOOP ;
12 END find_min ;
```

Procedure Calls

- **Cause execution of procedure body**
- **Can be used in sequential or concurrent statement areas**
- **Occur at or below level of procedure declaration**
- **Format:**

  procedure_name (passing_parameter_list) ;

- **Example:**

```
find_min (array_1, out_1, old_1) ;
```


Passing Parameters List

- **Specifies objects from and to which values pass**
- **Association by:**
  - **Position**
  - **Name**
- **Values associated with “in” parameters are “protected” from change**

ulogic_inverter

```
 1 LIBRARY ieee ;
 2 USE ieee.std_logic_1164.ALL ;
 3
 4 ENTITY invert_example IS
 5   PORT(in_a :  IN std_ulogic ;
 6        out_b: OUT std_ulogic ) ;
 7 END invert_example ;
 8
 9 ARCHITECTURE alg OF invert_example IS
10  PROCEDURE ulogic_inverter
11    (SIGNAL bit_in :  IN std_ulogic ;
12     SIGNAL bit_out: OUT std_ulogic ) IS
13  BEGIN
14    IF bit_in = '1' THEN
15      bit_out <= '0' ;
16    ELSIF bit_in = '0' THEN
17      bit_out <= '1' ;
18    ELSE
19      bit_out <= 'X' ;
20    END IF ;
21  END ulogic_inverter ;
22 BEGIN
23   ulogic_inverter (in_a, out_b) ;
24 END alg ;
```


Textual Input and Output

- **Provides method for reading from and writing to text files**
- **Text files are "plain text" files such as ASCII**
- **Plain text files:**
  - **Consist of lines of characters**
  - **Lines end with a carriage return**
- **Textio package:**
  - **Contents specified by IEEE**
  - **Declares types and subprograms for file manipulation**

Package textio

- **Files are frequently used with test benches to provide a source of test data and to provide storage of test results!**
- **VHDL provides a standard TEXTIO package that can be used to read or write lines of text from or to a file!**

```
-- Package TEXTIO as defined in Chapter 14
-- of the IEEE Standard VHDL
-- Language Reference Manual (IEEE Std. 1076-1987)
        use std.textio.all;
```


Declaring Files

- **Files are declared in declarative regions**
- **File declaration identifies:**
  - **Logical name of the file**
  - **Type of file**
  - **Mode of file (read_mode, write_mode, append_mode)**
  - **Relative or absolute file path name**

Declaring Files

- **Format:**

  FILE logical_file_name : type OPEN mode IS
  “physical/path/file_name”;-- VHDL ‘93 version

  Example:

  FILE test_data : text OPEN read_mode IS
  “c:\user\training\foo_file”;

File Pointer

- **A call of the form**

  ENDFILE(logical_file_name)

  returns a TRUE if the file pointer is at the end of the file

- **Example:**

  WHILE NOT (ENDFILE(test_data))

  LOOP

  -- reading or writing to file

  END LOOP;

Reading/Writing Files

- **Information is transferred to and from files by line**
- **Read a line of text from logical_file_name and place it in buffer (variable) of type *line* named line_name**
- **Write buffer to a line of text within logical_file_name**
- **Format:**

READLINE (logical_file_name, line_name);

WRITELINE (logical_file_name, line_name);

Reading/Writing Files

- **Read a object from the buffer**
- **Write a object to the buffer**
- **Format:**

READ (line_name, object_name);

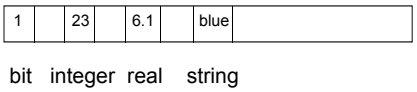
WRITE (line_name, object_name);

Processing Files

- **Include textio package**

  USE std.textio.ALL;

- **A file can be opened for read or write but not for both at the same time**
- **A line from a file can contain one of more objects separated by a space**
- **Each object in a line must be read before a new line is read**
- **Fill a line with objects before writing the line to a file**



File Example

```
USE std.textio.ALL ;
ENTITY example IS
        PORT (control : IN bit );
END example ;

ARCHITECTURE behav OF example IS
BEGIN
   PROCESS
      FILE data_in : text OPEN read_mode IS
        "input_data.txt";
      FILE data_out : text OPEN write_mode IS
       "output_data.txt";
      VARIABLE in_line, out_line : LINE ;
      VARIABLE inverse : bit ;
      VARIABLE half : integer ;
   BEGIN
```

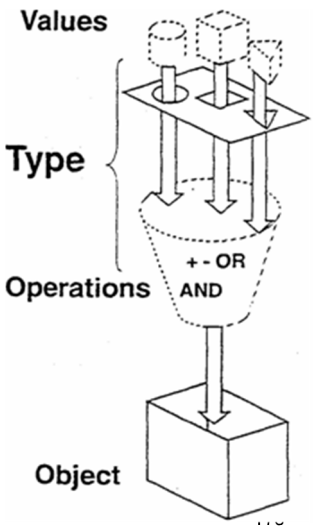
input_data.txt
1 3014

File Example

```
WAIT ON control;
      WHILE NOT ( ENDFILE (data_in) )
      LOOP
         READLINE ( data_in, in_line );
            READ ( in_line, inverse ) ;
            READ ( in_line, half ) ;
               inverse := NOT inverse ;
               half := half / 2 ;
            WRITE ( out_line, inverse ) ;
            WRITE ( out_line, half ) ;
         WRITELINE ( data_out, out_line ) ;
      END LOOP;
   END PROCESS;
END behav;
```


Types

- **Type determines the values an object can assume and operations that can be performed on it.**



Integer Types

- **“Integers” are the unbounded set of positive and negative whole numbers**
- **32-bit limitation restricts range**
- **Upper and lower range constraints must be integer numbers**
- **Declaration format:**

  TYPE type_name IS RANGE range_constraint ;
- **Predefined integer type:**

  TYPE integer IS RANGE -2147483648 TO 2147483647 ;

Range

- **Identifies subset of values**
- **May be used with type declarations or object declaration**
- **Format:**
  RANGE begin direction end
- **“Direction may be**
  - Ascending – TO
  - Descending – DOWNTO
- **Examples:**

```
TYPE day IS RANGE 1 TO 31 ;
TYPE voltage IS RANGE 12 DOWNTO -12  ;
SIGNAL in_volts : voltage RANGE 5 DOWNTO 0 ;
```

Floating Point Types

- **“Floating Points” are the unbounded set of positive and negative numbers that contain a decimal point**
- **32-bit limitation restricts range**
- **Upper and lower range constraints must contain a decimal point**
- **Declaration format:**

  TYPE type_name IS RANGE range_constraint ;
- **Predefined floating point type:**

  TYPE real IS RANGE -1.0E+38 TO 1.0E38 ;

Enumeration Types

- **List of identifiers or character literals**
- **Identifiers follow standard naming rules**
- **Character literals are all upper and lower case alpha characters, digits, and special characters, enclosed in single quotes**
- **Declaration format:**

  TYPE type_name IS (enumeration_ident_list) ;
- **Predefined enumeration types:**

```
TYPE bit IS (’0’, ’1’) ;
TYPE boolean IS ( false, true ) ;
TYPE severity_level IS (note, warning, error, failure);
TYPE character IS (’a’, ’b’, ’c’, . . .) ;
```

Physical Types

- **Describes objects in terms of a base unit, multiples of a base unit, and a specified range**
- **Range restricted to**
  -9223372036854775808 TO 9223372036854775807
- **Declaration format:**

```
TYPE type_name IS RANGE range_constraints
  UNITS
    base_unit ;
    [multiples ; ]
  END UNITS ;
```

- **Predefined physical type:**

```
TYPE time IS RANGE -9223372036854775808 TO
                    9223372036854775807
    UNITS
      fs ;
      ps = 1000 fs ;
      ns = 1000 ps ;
        .       .
        .
      hr = 60 min ;
    END UNITS ;
```


Scalar Subtypes

- **Subsets of specified types**
- **Range constraints must be within defining type's range**
- **Declaration format:**

  SUBTYPE name IS type_name
  RANGE constraints ;

- **Predefined scalar subtypes:**

```
SUBTYPE natural IS integer RANGE 0 TO 2147483647 ;
SUBTYPE positive IS integer RANGE 1 TO 2147483647 ;
```

Array Types

- **One or more dimensions**
- **Values referenced by indices**
- **Declaration format:**

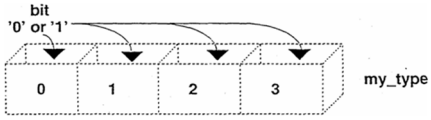
```
TYPE array_type_name IS ARRAY
   (range_constraint) OF type;
```

- **Predefined array types:**

```
TYPE string IS ARRAY (positive RANGE <>) OF character;
TYPE bit_vector IS ARRAY (natural RANGE <>) OF bit ;
```

- **Example:**

```
TYPE my_type IS ARRAY (integer RANGE 0 TO 3) OF bit ;
```



Array Range

- **Constrained or unconstrained**
- **Boundaries of constrained array are stated:**

```
TYPE array_1 IS ARRAY (integer RANGE -10 TO 25) OF bit;
```

- **Boundaries of unconstrained array are left open:**

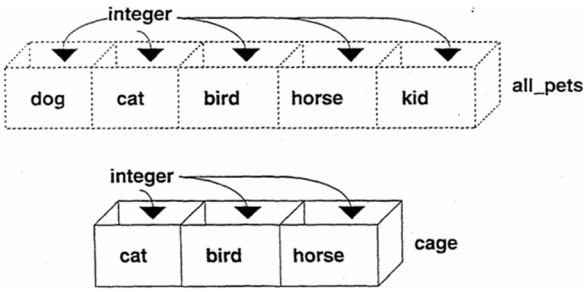
```
TYPE array_2 IS ARRAY (integer RANGE <>) OF bit ;
```

- **Boundaries can be enumerated types:**

```
TYPE pet IS (dog, cat, bird, horse, kid) ;


TYPE all_pets IS ARRAY (pet RANGE <>) OF integer ;
 SIGNAL cage : all_pets (cat TO horses) ;
```

Array Range

Example



Array Subtypes

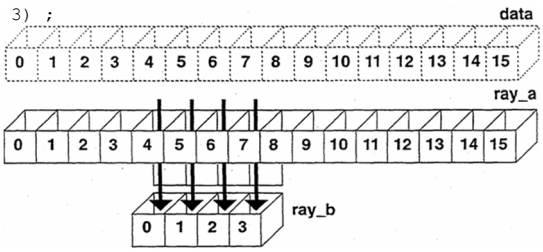
- **Subsets of specified array types**
- **The type that a subtype is based on must be an unconstrained array**
- **Declaration format:**
  SUBTYPE name IS
  array_name (range_constraint)
- **Examples:**

```
TYPE data IS ARRAY (natural RANGE <>) OF bit;
SUBTYPE low_range IS data(0 TO 7) ;
SUBTYPE high_range IS data(8 TO 15) ;
```

Array Slices

- **Consecutive positions of one-dimensional arrays**
- **Created by reference**
- **Example:**

```
PROCESS (a)
  TYPE size IS RANGE 0 TO 15 ;
  TYPE data IS ARRAY (size RANGE <>) OF bit;
  VARIABLE ray_a : data (0 TO 15) ;
  VARIABLE ray_b : data (0 TO 3) ;
BEGIN
  ray_b := ray_a(4 TO 7);
```



Record Types

- **It groups objects of different types together as a single object**
- **Any number of elements that can be scalars or composites**
- **Declaration Format:**

```
TYPE record_type_name IS
  RECORD
    identifier_list : type ;
      [ . . . ]
    END RECORD ;
```

- **Example:**

```
TYPE op IS (add, sub) ;
TYPE instruct IS
  RECORD    opcode : op ;
            source : integer RANGE 0 TO 7 ;
            dest : integer RANGE 0 TO 15 ;
  END RECORD ;
  . . .
```

Accessing a Record Object

- **Record object elements referenced by name**
- **Example:**

```
TYPE op IS (add, sub) ;
 TYPE instruct IS
   RECORD opcode : op ;
          source : integer RANGE 0 TO 7 ;
            dest : integer RANGE 0 TO 15
   END RECORD ;
   . . .

 PROCESS
   VARIABLE inst : instruct := (add, 7, 15) ;
 BEGIN
   inst.dest := 5 ;
   IF inst.opcode = sub THEN
   . . .
```


Predefined Type Attributes

- **TYPE ex IS (hold, clear, start, stop);**
  SUBTYPE subset IS ex RANGE clear TO start ;
- **type'HIGH ex'HIGH --> stop**
- **type'LOW ex'LOW --> hold**
- **type'RIGHT ex'RIGHT --> stop**
- **type'LEFT ex'LEFT --> hold**
- **type'RIGHTOF(x) ex'RIGHTOF(clear) --> start**

Predefined Type Attributes

- type'LEFTOF(x) ex'LEFTOF(clear) --> hold
- type'PRED(x) ex'PRED(clear) --> hold
- type'SUCC(x) ex'SUCC(clear) --> start
- type'POS(x) ex'POS(clear) --> 1
- type'VAL(x) ex'VAL(0) --> hold

Strongly Typed

- Objects may assume only values of their type
- Only operations defined for a type may be applied to objects of that type

Arithmetic Operations

- **Predefined binary operators:**

**	exponentiation
*	multiplication
/	division
MOD	modulus
REM	remainder
+	addition
-	subtraction

- **Predefined unary operators:**

+	sign operator plus
-	sign operator minus
ABS	absolute value

Exponentiation **

- **Format:**

  left_operand ** right_operand

- **Left_operand**

  integer or floating point object/number

- **Right_operand**

  integer object/number

- **Result:**

  same type as left_operand

- **Negative right_operands used only with floating point left_operands**

Multiplication * and Division /

- **Format:**
  left_operand * right_operand
  left_operand / right_operand
- **Result type:**

left_operand	right_operand	result
integer	integer physical	integer physical
floating point	floating point physical	floating point physical
physical	integer floating point	physical physical
(Division Only)		
physical	physical	integer

MOD and REM

- **Remainder operators defined only for integer types**
- **Format:**
  left_operand MOD right_operand
  left_operand REM right_operand
- **(A REM B) result with same sign as A and |result| < |B|**
  9 REM 5 = 4 9=1*5+4
  -8 REM 6 = -2 -8=-1*6-2
- **(A MOD B) result with same sign as B and |result| < |B|**
  9 MOD 5 = 4 9=1*5+4
  -8 MOD 6 = 4 -8=-2*6+4

+ and -

- **Perform addition (+) and subtraction (-)**
- **Format:**

  left_operand + right_operand
  left_operand - right_operand

- **Left_operand**

  integer, floating point, or physical object/value

- **Right_operand**

  same type as left operand

- **Result:**

  same type as left_operand

Unary + and -

- **Sign operators:**
  - **+ (identify function)**
  - **- (negation function)**
- **Format:**

  + right_operand
  - right_operand

- **Right_operand**

  integer, floating point, or physical object/value

- **Result:**

  same type as right_operand

ABS

- **Returns positive value of operand**
- **Format:**
  ABS(right_operand)
- **Right_operand:**
  integer, floating point, or physical object/value
- **Result:**
  same type as right operand

Subtype Calculations

- **Example**

```
power: PROCESS (clk)
    TYPE num IS RANGE 1 TO 20 ;
    SUBTYPE low_rng IS num RANGE 1 TO 10;
    SUBTYPE high_rng IS num RANGE 10 TO 20 ;
    VARIABLE x : low_rng;
    VARIABLE y, z : high_rng;
BEGIN
    z := x + y;
```

- **END PROCESS power;**

Arithmetic Precedence

- **Order in which calculations are performed**
- **Expressions are evaluated left to right in the following order**

Symbol	Evaluated
** ABS	first pass
* / MOD REM	second pass
sign (unary +, -)	third pass
+ -	fourth pass

Converting Types

- **Changing values from one type to another**
- **Strong typing does not allow implicit value conversions**
- **Example of illegal usage:**

```
PROCESS (clk)
        VARIABLE int : integer RANGE 0 TO 10 ;
        VARIABLE flt : real RANGE 0.0 TO 10.0 ;
  BEGIN
        flt := int ; - - ILLEGAL
  END PROCESS;
```

- **Explicit conversion is necessary:**
  - **typemark conversion**
  - **user-created conversion**

Typemark Conversion

- **Converts from one closely related type to another**
- **Format:**
  typemark (object)
- **Typemark:**
  name of type to be converted to
- **Example**

```
PROCESS (clk)
        VARIABLE int_er : integer RANGE 0 TO 10 ;
        VARIABLE flt_er : real RANGE 0.0 TO 10.0 ;
        VARIABLE tot : REAL ;
BEGIN
        tot := real(int_er) + flt_er ;
END PROCESS;
```


Valid Typemark Conversions

- **Closely related types:**

From:		To:
integer		integer
floating point	→	floating point
integer		floating point
floating point		integer
array type*		array type*

Valid Typemark Conversions

- **Example:**

```
PROCESS (clk)
        TYPE a IS RANGE 10.0 TO 12.0 ;
        TYPE b IS RANGE 10.0 TO 30.0 ;
        VARIABLE hat : a ;
        VARIABLE pat : b RANGE 10.0 TO 15.0 ;
        VARIABLE tot : b ;
BEGIN
        tot := b(hat) + pat ;
END PROCESS
```


User Created Conversions

- **Functions created by user**
- **Offer wider range of conversion possibilities**
- **Example:**

```
FUNCTION int_to_flt (in_val : integer RANGE 1 TO 4)
        RETURN real IS
BEGIN
        CASE in_val IS
                WHEN 1 => RETURN 1.0;
                WHEN 2 => RETURN 2.0;
                WHEN 3 => RETURN 3.0;
                WHEN 4 => RETURN 4.0;
        END CASE ;

END int_to_flt ;
```

- **Calling**

```
float_sum <= int_to_flt(int_val) + float_val;
```

Overloaded Operators

- VHDL arithmetic operators + and – are defined to operate on integer or floating point, but not on bit_vectors!
- Operator overload means to extend the definition of the operator to other data types!
- *IEEE.numeric_bit* package provides overloaded arithmetic operators for bit_vectors using unsigned or signed type!
- Creation of overloaded operators is possible by user defined VHDL packages!

Package

- Shareable collections of declarations made outside models
- Allow common use of:
  - Subprograms
  - Types, subtypes
  - Files
  - Constants, signals
  - Aliases
  - Attributes
  - Components
- Consist of:
  - Declaration
  - Body (body required only if declaring subprograms)
- Package must be compiled to be accessible

Package Declaration

- **Format:**

  PACKAGE package_name IS
  --identifier declarations
  END package_name ;

- **Allowable declarations:**
  - **Subprograms**
  - **Types, subtypes**
  - **Files**
  - **Constants, signals**
  - **Aliases**
  - **Attribute specifications**
  - **Component specifications**
  - **Disconnection specifications**
  - **(Use clause)**

Example:

```
PACKAGE example IS
 TYPE tj IS ('T', 'J') ;
 CONSTANT pi : real := 3.14159 ;
 END example ;
```


Package Body

- **Describes implementation of subprograms**
- **Format:**

  PACKAGE BODY package_name IS
  -- declarations, subprogram bodies
  END package_name ;

- **Package_name must be same as name used in package declaration**
- **Declarations within body are not visible unless stated in package declaration**
- **Example:**

```
PACKAGE BODY example IS
   FUNCTION mean (a, b, c : real )
    RETURN real IS
    BEGIN
     RETURN (a + b + c)/3.0 ;
   END mean ;
 END example ;
```

Using Packages

- **Format:**

  USE library_name.package_name.identifier_name ;

- **To specify the use of all declarations made in a package, use .ALL for identifier_name**

```
USE things.example.ALL;-- user defined package

USE IEEE.std_logic_1164.ALL ;-- predefined package
```


Package/Library Example

```
 1 PACKAGE func_pkg IS
 2   FUNCTION mean (a, b, c: real)
 3    RETURN real;
 4 END func_pkg ;
-------------------------------------------------
 1 PACKAGE BODY func_pkg IS
 2   FUNCTION mean (a, b, c: real)
 3    RETURN real IS
 4   BEGIN
 5     RETURN (a+b+c)/3.0;
 6   END mean ;
 7 END func_pkg;
-------------------------------------------------
 1 LIBRARY things;
 2 USE things.func_pkg.ALL;
 3
 4 ENTITY average IS
 5   PORT (in_1, in_2, in_3:  IN real ;
 6              signal_out: OUT real );
 7 END average ;
 8
 9 ARCHITECTURE pkg_example OF average IS
10 BEGIN
11   signal_out <= mean(in_1, in_2, in_3);
12  END pkg_example ;
```

Predefined Packages

Packages out of IEEE library

```
library IEEE;
```

- **USE IEEE.numeric_bit.ALL** ;-- arithm. Operators for bit_vector
  -- Two numeric types are defined: UNSIGNED and SIGNED
  -- Signed vectors are represented in two's complement form

- **USE IEEE.std_logic_1164.ALL** ;-- IEEE 9-Valued Logic

- **USE IEEE.numeric_std.ALL** ;--arithm. Operators for std_logic_vector
  -- Two numeric types are defined: UNSIGNED and SIGNED
  -- Signed vectors are represented in two's complement form

Packages out of Standard library

- **USE std.textio.ALL** ;-- for file input and output

Building Blocks

Entity Declaration

Description	Example

entity entity_name is
port (
[signal] identifier {, identifier}: [mode] signal_type
{; **[signal]** identifier {, identifier}: [mode] signal_type});
end [entity] [entity_name];
	entity register8 is
	port (
	clk, rst, en: in std_logic;
	data: in std_logic_vector(7 **downto** 0);
	q: out std_logic_vector(7 **downto** 0);
	end register8;

Entity Declaration with Generics

Description	Example

entity entity-name is
generic (
[signal] identifier {, identifier}: [mode] signal-type
[:= static_expression]
{; **[signal]** identifier {, identifier}: [mode] signal_type
[:= static_expression] }
);
port (
[signal] identifier {, identifier}: [mode] signal_type
{; [signal] identifier {, identifier}: [mode] signal_type}
);
end [entity] [entity_name] ;
	entity register_n is
	generic (
	width: integer := 8);
	port (
	clk, rst, en: **in** std_logic;
	data: **in** std_logic_vector(width-1 **downto** 0);
	q: **out** std_logic_vector(width-1 **downto** 0))
	end register_n;

Architecture Body

Description	Example

architecture architecture_name **of** entity **is**
type_declaration
| signal _declaration
| constant_declaration
| component_declaration
| alias_declaration
| attribute_specification
| subprogram_body
begin
{process_statement
| concurrent_signal_assignment_statement
| component_instantiation_statement
| generate_statement
end [architecture] [architecture_name];
	architecture archregister8 of register8 is
	begin
	process (rst, clk)
	begin
	if (rst = '1') **then**
	q <= (others => 0);
	elseif (clk'event **and** clk = '1') **then**
	if (en = '1') **then**
	q <= data;
	else
	q <= q;
	end if;
	end if;
	end process;
	end archregister8**;**
	
	architecture archfsm of fsm is
	type state)type is (st0, st1, st2);
	signal state: state_type;
	signal y, z: std_logic;
	begin
	process begin
	wait until clk' = '1';
	case state is
	when st0 =>
	state <= st1;
	y <= '1';
	when st1 =>
	state <= st2;
	z <= '1';
	when others =>
	state <= st3;
	y <= '0';
	z <= '0';
	end case;
	end process;
	end archfsm;

Declaring a Component

Description	Example

component component_name
port (
[signal] identifier {, identifier}: [mode] signal_type
{; **[signal]** identifier {, identifier}: [mode] signal_type]);
end component [component_name] ;
	component register8
	port (
	c1k, rst, en: **in** std_logic ;
	data: **in** std_1ogic_vector(7 **downto** 0);
	q: **out** std_logic_vector(7 **downto** 0));
	end component;

Declaring a Component with Generics

Description	Example

component component_name
generic (
[**signal**] identifier {, identifier}: [mode] signal_type
[: =static_expression]
{; [**signal**] identifier {, identifier}: [mode] signal_type
[: =static_expression]);
port (
[**signal**] identifier {, identifier}: [mode] signal_type
{; [**signal**] identifier {, identifier}: [mode] signal_type);
end component [component_name];
	component register8
	generic (
	width: integer := 8);
	port (
	clk, rst, en: **in** std_logic;
	data: **in** std_1ogic_vector(width-1 **downto** 0);
	q: **out** std_logic_vector (width-l **downto** 0));
	end component;

Component Instantiation (named association)

Description	Example

instantiation_label:
component_name
port map (
port_name => signal_name
| expression
| variable_name
| **open**
{, port_name => signal_name
| expression
| variable_name
| **open**});
	architecture archreg8 of reg8 is
	signal clock, reset, enable: std_logic;
	signal data-in, data-out: std_logic_vector(T **downto** 0);
	begin
	first_reg8:
	register8
	port map (
	clk => clock,
	rst => reset,
	en => enable,
	data => data_in,
	q => data_out);
	end archreg8;

Component Instantiation with Generics (named association)

Description	Example

Instantiation_label:
Component_name
generic map(
generic_name => signal_name
| expression
| variable_name
| **open**
{, generic_name => signal_name
| expression
| variable_name
| **open})**
port map (
port_name => signal_name
| expression
| variable_name
| **open**
{, port_name => signal_name
| expression
| variable_name
| **open**});
	architecture archreg5 **of** reg5 **is**
	signal clock, reset, enable: std_logic;
	signal data_in, data_out: std_logic_vector(7 **downto** 0);
	begin
	first_reg5:
	register_n
	generic map (width => 5) --no semicolon here
	port map (
	clk => clock ,
	rst => reset,
	en => enable,
	data => data-in,
	q => data_out);
	end archreg5;

Component Instantiation (positional association)

Description	Example
instantiation_label: component_name port map (signal_name | expression | variable_name | open {, signal_name | expression | variable_name | open});	**architecture** archreg8 **of** reg8 **is** **signal** clock, reset, enable: std_logic; **signal** data_in, data_out: std_logic_vector(7 **downto** 0); **begin** **first_reg8:** register8 **port map** (clock, reset, enable, data_in, data_out); **end** archreg8;

Component Instantiation with Generics (positional association)

Description	Example

instantiation_lable:
component_name
generic map (signal_name | expression
| variable_name | open
{, signal_name | expression
| variable_name | open})
port map (signal_name | expression
| variable_name | open
{, signal_name | expression
| variable_name | open});
	architecture archreg5 of reg5 is
	signal clock, reset, enable: std_logic;
	signal data_in, data_out: std_logic_vector(7 downto 0);
	begin
	first_reg5:
	register_n
	generic map (5)
	port map (clock, reset, enable, data_in, data_out);
	end archreg5;

Concurrent Statements

Boolean Equations

Description	Example
relation { and relation } | relation { or relation } | relation { xor relation } | relation { nand relation } | relation { nor relation }	v<= (a and b and c) or d; --parenthesis req'd w/ 2-level logic w <= a or b or c; x <= a xor b xor c; y <= a nand b nand c; z <= a nor b nor c;

When-else Conditional Signal Assignment

Description	Example
{expression when condition else} expression;	x<= '1' when b = c else '0'; x<= j when state = idle else k when state = first_state else l when state = second_state else m when others;

With-select-when Select Signal Assignment

Description	Example

with selection_expression select
{identifiers <= expression when
identifier | expression | discrete_range | others,}
identifier <= expression when
identifier | expression | discrete_range | others;
	architecture archfsm of fsm is
	type state_type is (st0, st1, st2, st3, st4, st5, st6, st7, st8);
	signal state: state_type;
	signal y, z: std_logic_vector(3 downto 0);
	begin
	with state select
	x<= "0000" when st0 | st1; -- st0 "or" st1
	"0010: when st2 | st3;
	z when st4;
	z when others;
	end archfsm;

Generate Scheme for Component Instantiation or Equations

Description	Example

generate_label: (for identifier in discrete_range)
| (if condition) generate
{concurrent_statement}
end generate [generate_label];
	g1: for i in 0 to 7 generate
	reg1: register8 port map (clock, reset, enable,
	data_in(i), data_out(i);
	g2: for j in 0 to 2 generate
	a(j) <= b(j) xor c(j);
	end generate g2;

Sequential Statements

Process Statement

Description	Example

[process_lable:]
process (sensitivity_list)
  type_declaration
| constant_declaration
| variable_declaration
|alias_declaration}
begin
{wait_statement
| signal_assignment_statement
| variable_assignment_statement
| if_statement
| case_statement
| loop_statement
end process [process_label];
	my_process
	process (rst, clk)
	  constant zilch : std-logic_vector(7 downto 0) :=
	"0000_0000";
	begin
	  wait until clk = '1';
	  if (rst = '1') then
	  q <= zilch;
	  elsif (en = '1') then
	  q <= data;
	  else
	  q <= q;
	  end if
	end my_process;

if-then-else Statement

Description	Example

if condition then sequence_of_statements
{elsif condition then sequence_of_statements}
[else sequence_of_statements}
end if;
	if (count = "00") then
	  a <= b;
	elsif (count = "10") then
	  a <= c;
	else
	  a <= d;
	end if;

case-when Statement

Description	Example

case expression is
{when identifier | expression | discrete_range | others =>
  sequence_of_statements}
end case;
	case count is
	  when "00" =>
	  a <= b;
	  when "10 =>
	  a <= c;
	  when others =>
	  a <= d;
	end case;

for-loop Statement

Description	Example

[loop_label:]
  for identifier in discrete_range loop
  {sequence_of_statements}
  end loop [loop_label];
	my_for_loop
	for i in 3 downto 0 loop
	  if reset(i) = '1' then
	  data_out(i) := '0';
	  end if;
	end loop my_for_loop;

while-loop Statement

Description	Example

[loop_label:]
  while condition loop
  {sequence_of_statements}
  end loop [loop_label];
	count := 16;
	my_while_loop:
	  while (count > 0) loop
	  count:= count – 1;
	  Result <= result + data_in;
	end loop my_while_loop;

Describing Synchronous Logic Using Processes

No Reset (Assume clock is of type std_logic)

Description	Example

[process_label:]
process (clock)
begin
  if clock'event and clock = '1' then --or rising_edge
  synchronous_signal_assignment_statement;
  end if;
end process [process_label];
----------or----------
[process_label:]
process
begin
  wait until clock = '1';
  synchronous_signal_assignment_statement;
end process [process_label];
	reg8_no_reset:
	process (clk)
	begin
	  if clk'event and clk = '1' then
	  q <= data;
	  end if;
	end process reg8_no_reset;
	----------or----------
	reg8_no-reest:
	process
	begin
	  wait until clock = '1';
	  q <= data;
	end process reg8_no_reset;

Synchronous Reset

Description	Example

[process_label:]
process (clock)
begin
  if clock'event and clock = '1' then
  if synch_reset_signal = '1' then
  synchronous_signal_assignment_statement;
  else
  synchronous_signal_assignment_statement;
  end if;
  end if;
end process [process_label];
	reg8_sync_reset:
	process (clk)
	begin
	  if clk'event and clk = '1' then
	  if sync_reset = '1' then
	  q <= "0000_0000";
	  else
	  q <= data;
	  end if;
	  end if;
	end process;

Asynchronous Reset or Preset

Description	Example

[process_label:]
process (reset, clock)
begin
  if reset = '1' then
  asynchronous_signal_assignment_statement;
  elsif clock'event and clock = '1' then
  synchronous_signal_assignment_statement;
  end if;
end process [process_label];
	reg8_async_reset:
	process (asyn_reset, clk)
	begin
	  if async_reset = '1' then
	  q <= (others => '0');
	  elsif clk'event and clk = '1' then
	  q <= data;
	  end if;
	  end process reg8_async_reset;

Asynchronous Reset and Preset

Description	Example

[process_label:]
process (reset, preset, clock)
begin
  if reset = '1' then
  asynchronous_signal_assignment_statement;
  elsif preset = '1' then
  asynchronous_signal_assignment_statement;
  elsif clock'event and clock = '1' then
  synchronous_signal_assignment_statement;
  end if;
end process [process_label];
	reg8_async:
	process (asyn_reset, async_preset, clk)
	begin
	  if async_reset = '1' then
	  q <= (others => '0');
	  elsif async_preset = '1' then
	  q <= (others => '1');
	  elsif clk'event and clk = '1' then
	  q <= data;
	  end if;
	end process reg8_async;

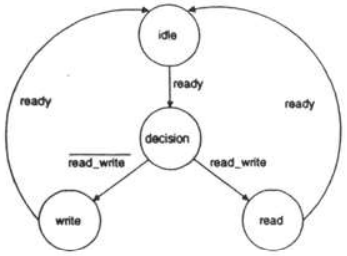
Conditional Synchronous Assignment (enables)

Description	Example

[process_label:]
process (reset, clock)
begin
  if reset = '1' then
  asynchronous_signal_assignment_statement;
  elsif clocl'event and clock = '1' then
  if enable = '1' then
  synchronous_signal_assignment_statement;
  else
  synchronous_signal_assignment_statement;
  end if;
  end if;
end process [process_label];
	reg8_sync_assign:
	process (rst, clk)
	begin
	  if rst = '1' then
	  q <= (others => '0');
	  elsif clk'event and clk = '1' then
	  if enable = '1' then
	  q <= data;
	  else
	  q <= q;
	  end if;
	  end if;
	end process reg8_sync_assign;

Translating a State Flow Diagram to a Two-Process FSM Description

Description	Example


state | outputs oe | we
idle 0 0
decision 0 0
write 0 1
read 1 0
	architecture state_machine of example is
	type StateType is (idle, decision, read, write);
	signal present_state, next_state : StateType;
	begin
	state_comb:process(present_state, read_write, ready) begin
	case present_state is
	when idle => oe <= '0' ; we <= '0';
	if ready = '1' then
	next_state <= decision;
	else
	next_state <= idle;
	end if;
	when decision => oe <= '0' ; we <= '0';
	if (read_write = '1') then
	next_state <= read;
	else -- read_write='0'
	next_state <=write;
	end if;
	when read => oe <= '1' ; we <='0';
	if (read = '1') then
	next_state <= idle;
	else
	next_state <= read;
	end if;
	when write => oe <= '0' ; we <= '1';
	if (ready = '1') then
	next_state <= idle;
	else
	next_state <= write;
	end if;
	end case;
	end process state_comb;
	
	state_clocked:process(clk) begin
	if (clk'event and clk = '1') then
	present_state <= next_state;
	end if;
	end process state_clocked;
	end state_machine;

Data Objects

Signals

Description	Example

• Signals are the most commonly used data object in synthesis designs.
• Nearly all basic designs, and many large designs as well, can be fully described using signals as the only kind of data object.
• Signals have projected output waveforms.
• Signal assignments are scheduled, not immediate; they update projected output waveforms.
	architecture archinternal_counter of internal_counter is
	signal count, data:std_logic_vector(7 downto 0);
	begin
	process(clk)
	begin
	if (clk'event and clk = '1') then
	if en = '1' then
	count <= data;
	else
	count <= count + 1;
	end if;
	end if;
	end process;
	end archinternal_counter;

Constants

Description	Example

Constants are used to hold a static value; they are typically used to improve the readability and maintenance of code.	my_process:
	process (rst, clk)
	constant zilch : std_logic_vector(7 downto 0) := "0000_0000";
	begin
	wait until clk = '1';
	if (rst = '1') then
	q <= zilch;
	elsif (en = '1') then
	q <= data;
	else
	q <= q;
	end if;
	end my_process;

bit and bit_vector

Description	Example

• Bit values are: '0' and '1'.
• Bit_vector is an array of bits.
• Pre-defined by the IEEE 1076 standard.
• This type was used extensively prior to the introduction and synthesis-tool vendor support of std_logic_1164.
• Useful when metalogic values not required.
	signal x: bit;
	. . .
	if x = '1' then
	state <= idle;
	else
	state <= start;
	end if;

Boolean

Description	Example

• Values are TRUE and FALSE.
• Often used as return value of function
	signal a: boolean;
	. . .
	if x = '1' then
	state <= idle;
	else
	state <= start;
	end if;

Integer

Description	Example

• Values are the set of integers.
• Data objects of this type are often used for defining widths of signals or as an operand in an addition or subtraction.
• The types std_logic_vector and bit_vector work better than integer for components such as counters because the use of integers may cause "out of range" run-time simulation errors when the counter reaches its maximum value.
	entity counter_n is
	generic (
	width: integer := 8);
	port (
	clk, rst, in std_logic;
	count: out std_logic_vector(width-1 downto 0));
	end counter_n;
	. . .
	process(clk)
	begin
	if (rst = '1') then
	count <= 0;
	elsif (clk'event and clk = '1') then
	count <= count +1;
	end if;
	end process;

Enumeration Types

Description	Example

• Values are user-defined
• Commonly used to define states for a state machine.
	architecture archfsm of fsm is
	type state_type is (st0, st1, st2);
	signal state: state_type;
	signal y, z: std_logic;
	begin
	process
	begin
	wait until clk'event = '1';
	case state is
	when st0 =>
	state <= st2;
	y <= '1'; z <= '0';
	when st1 =>
	state <= st3;
	y <= '1'; z <= '1';
	when others =>
	state <= st0;
	y <= '0'; z <= '0';
	end case;
	end process;
	end archfsm;

Variables

Description	Example

• Variables can be used in processes and subprograms -- that is, in sequential areas only.
• The scope of a variable is the process or subprogram
• A variable in a subprogram does not retain its value between calls.
• Variables are most commonly used as the indices of loops or for the calculation of intermediate values, or immediate assignment.
• To use the value of a variable outside of the process or subprogram in which it was declared the value of the variable must be assigned to a signal
• Variable assignment is immediate, not scheduled
	architecture archloopstuff of loopstuff is
	signal data: std_logic_vector(3 downto 0);
	signal result: std_logic;
	begin
	process (data)
	variable tmp: std_logic;
	begin
	tmp := '1';
	for i in a'range downto 0 loop
	tmp := tmp and data(i);
	end loop;
	result <= tmp;
	end process;
	end archloopstuff;

Data Types and Subtypes

std_logic

Description	Example
• Values are: 'U', -- Uninitialized 'X', -- Forcing unknown '0', -- Forcing 0 '1', -- Forcing 1 'Z', -- High impedance 'W', -- Weak unknown 'L', -- Weak 0 'H', -- Weak 1 '-', -- Don't care • The standard multivalue logic system for VHDL model inter-operability. • A resolved type (i.e., a resolution function is used to determine the value of a signal with more than one driver). • To use must include the following two lines: library ieee; use ieee.std_logic_1164.all;	Signal x, data, enable: std_logic; . . . x <= data when enable = '1' else 'Z';

std_ulogic

Description	Example
• Values are: 'U', -- Uninitialized 'X', -- Forcing unknown '0', -- Forcing 0 '1', -- Forcing 1 'Z', -- High impedance 'W', -- Weak unknown 'L', -- Weak 0 'H', -- Weak 1 '-', -- Don't care • An unresolved type (i.e., a signal of this type may have only one driver). • Along with its subtypes, std_ulogic should be used over user-defined ability of VHDL models among synthesis and simulation tools. • To use must include the following two lines: library ieee; use ieee.std_logic_1164.all;	Signal x, data, enable: std_ulogic; . . . x <= data when enable = '1' else 'Z';

std_logic_vector and std_ulogic_vector

Description	Example

• Are arrays of types std_logic and std_ulogic.
• Along with its subtypes, std_logic_vector should be used over user-defined types to ensure interoperability of VHDL models among synthesis and simulation tools.
• To use must include the following two lines:
library ieee;
use ieee.std_logic_1164.all;
	signal mux: std_logic_vector (7 downto 0)
	. . .
	if state = address or state = ras then
	mux <= dram_a;
	else
	mux <= (others => 'Z');
	end if;

In, Out, buffer, inout

Description	Example

• In: Used for signals (ports) that are inputs-only to an entity.
• Out: Used for signals that are outputs – only and for which the values are not required internal to the entity.
• Buffer: Used for signals that are outputs but for which the values are required internal to the given entity. Caveat with usage: If the local port of the instantiated component is of mode buffer, then if the actual is also a port it must of mode buffer as well. For this reason some designers standardize on mode buffer.
• Inout: Used for signals tha are truly bidirectional. May also be used for signals for that are input-only or output-only, at the expense of code readability.
	Entity counter_4 is
	port (
	clk, rst, ld: in std_logic;
	term_cnt: buffer std_logic;
	count: inout std_logic_vector (3 down to 0));
	end counter_4;
	architecture archcounter_4 of counter_4 is
	signal int_rst: std_logic;
	signal int_count: std_logic_vector(3 downto 0));
	begin
	process(int_rst, clk)
	begin
	if(int_rst = '1') then
	int_count <= "0000";
	elseif (clk'event and clk='1') then
	if(ld = '1') then
	int_count <=count;
	else
	int_count <=int_count + 1;
	end if;
	end if;
	end process;
	term_cnt <= count(2) and count(0);
	--term_cnt is 3
	int_rst <=term_cnt or rst;
	-- resets at term_cnt
	count <=int_count when ld='0' else "ZZZZ";
	--count is bidirectional
	end architecture_4;

Operators

All operators of the same class have the same level of precedence. The classes of operators are listed here in the order of decreasing precedence. Many of the operators are overloaded in the std_logic_1164, numeric_bit, and numeric_std packages.
Miscellaneous Operators…page 164

Description	Example
• Operators: **, abs, not • The not operator is used frequently, the other two are rarely used for designs to be synthesized. • Predefined for any integer type (**), any numeric type (abs), and either bit and Boolean (not).	signal a, b, c:bit; … a <= not (b and c);

Multiplying Operators

Description	Example
• Operators: *, /, mod, rem. • The * operator is occassionaly used for multipliers; the other three are rarely used in synthesis. • Predefined for any integer type (*, /, mod, rem), and any floating point type (*,/).	variable a, b:integer range 0 to 255; . . . a <= b * 2;

Sign

Description	Example
• Operators: +,-. • Rarely used for synthesis. • Predefined for any numeric type (floating – point or integer).	variable a, b, c: integer range 0 to 255; . . . a <= - (b+2);

Adding Operators

Description	Example
• Operators: +,- • Used frequently to describe incrementers, decrementers, adders and subtractors. • Predefined for any numeric type.	signal count: integer range 0 to 255; . . . count <= count+1;

Shift Operators

Description	Example

• Operators: sll, srl, sla, sra, rol, ror.
• Used occasionally.
• Predefined for any one-dimensional array with elements of type bit or Boolean. Overloaded for std_logic arrays.
	signal a, b: bit_vector(4 downto 0);
	signal c: integer range 0 to 4;
	. . .
	a<=b sll c;

Relational Operators

Description	Example

• Operators: =, /=, <, <=, >, >=.
• Used frequently for comparisons.
• Predefined for any type (both operands must be of same type)
	signal a, b: integer range 0 to 255;
	signal agtb: std_logic;
	. . .
	if a >=b then
	agtb <= '1';
	else
	agtb <='0';

Logical Operators

Description	Example
• Operators: and, or, nand, nor, xor, xnor. • Used frequently to generate Boolean equations. • Predefined for types bit and Boolean. Std_logic_1164 overloads these operators for std_ulogic and its subtypes.	signal a, b, c:std_logic; . . . a<=b and c;